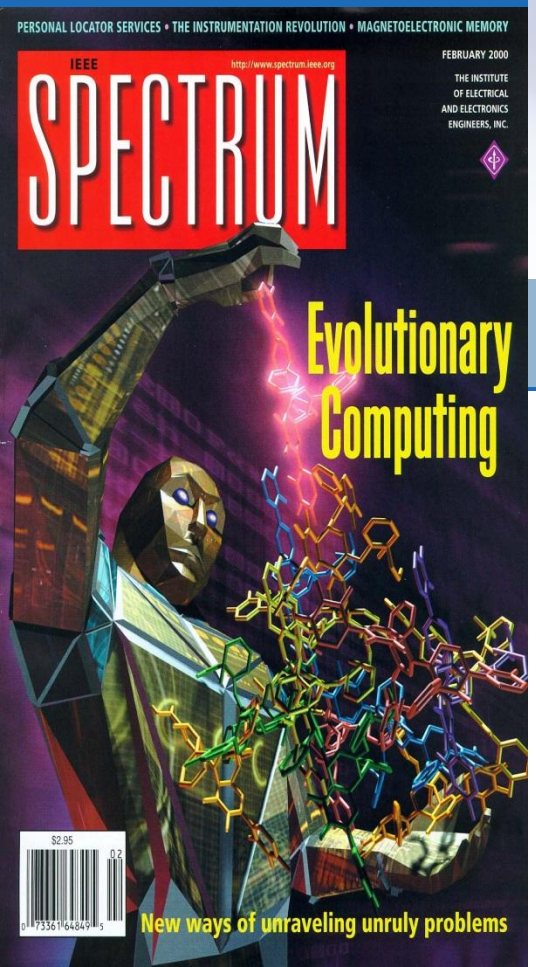
BIT.MLMR

Arthur Keith: I hold that individual liberties, so long as they do not endanger the unity and welfare of the community or tribe, have an evolutionary value.



Evolutionary Computation

Lecture 6







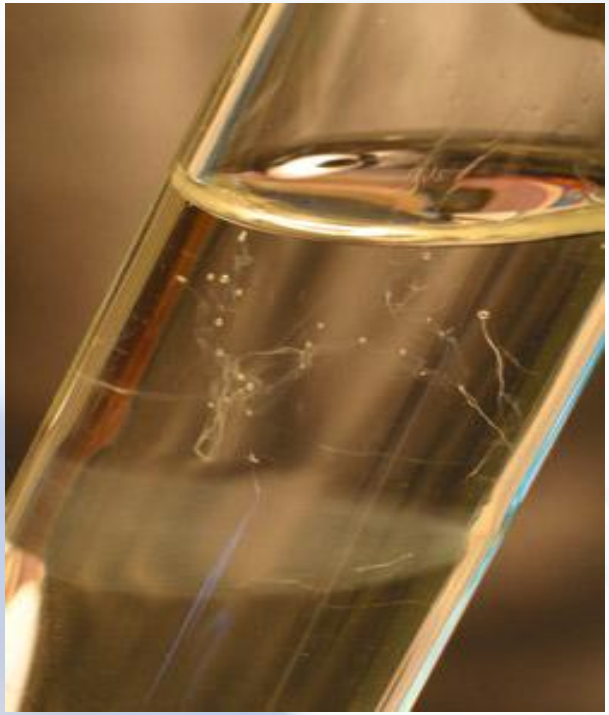
Adleman's Experiment

- **Hamilton Path Problem**
  (also known as the travelling salesperson problem)



Is there any Hamiltonian path from Darwin to Alice Spring?

Adleman's Experiment: Process



Formation of DNA strands.



Precipitation of more DNA strands in alcohol



Spooling the DNA with a metal hook or similar device

Outline

- **Biological Inspiration to Search**
- **DNA Computing**
- **What is Evolutionary Computation (EC)?**
- **Evolutionary Algorithms (EA)**
  - Genetic Algorithm
  - Genetic Programming
  - Evolutionary Programming
  - Evolutionary Strategies

Part Ⅰ: Biological Inspiration to Search

Q. What is the best problem solver (searcher) in the universe?

A. **The (human) brain**

that created "the wheel, New York, wars, and so on" (after Douglas Adams)

→ Neural Computation

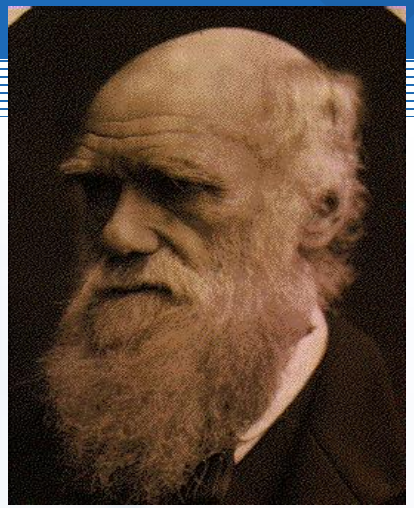
A. **The evolution mechanism**

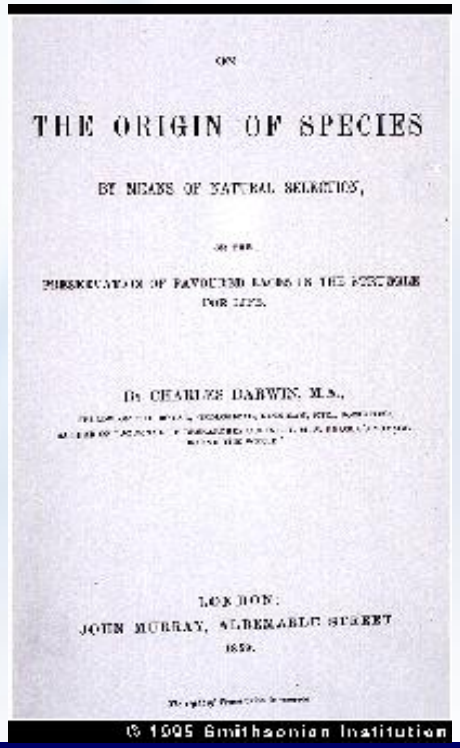
that created the human brain (after Darwin et al.)

→ Evolutionary Computation

Darwinian Evolution

- Works on population scale, not individual
- Survival of the fittest: Finite resources to yield competition, Those individuals that compete for the resources most effectively (fitness) have increased chance of reproduction (selection)




ON

THE ORIGIN OF SPECIES

BY MEANS OF NATURAL SELECTION,

OR THE

PRESERVATION OF FAVOURED RACES IN THE STRUGGLE FOR LIFE.

By CHARLES DARWIN, M.A.,

LONDON:
JOHN MURRAY, ALBEMARLE STREET
1859.

© 1995 Smithsonian Institution

Darwinian Evolution (Cont'd)

- **Diversity drives change: If phenotypic traits:**
  - Lead to higher chances of reproduction
  - Can be inherited

  then they will tend to increase in subsequent generations, leading to new combinations of traits.

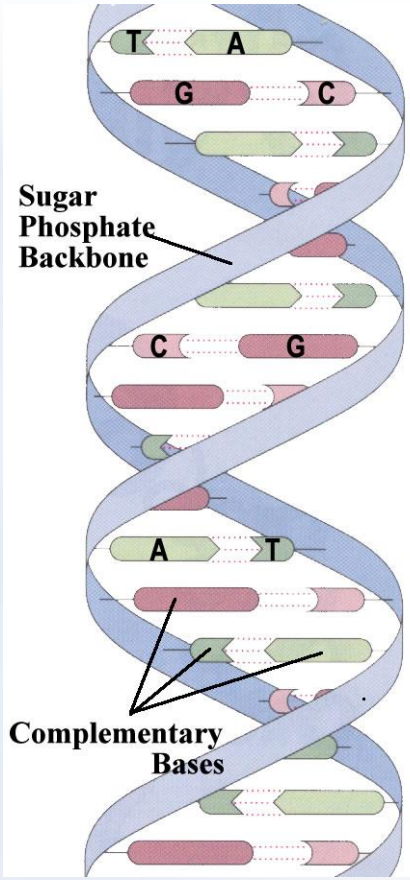
- **Chance plays a part**
  - Variation affects viability
  - Fittest don't always survive!

Natural Genetics

- The information required to build a living organism is coded in the DNA (Genotype) of that organism
- Small changes in the genotype lead to small changes in the organism (e.g., height, hair colour)
- Within a species, most of the genetic material is the same

Genes and the Genome

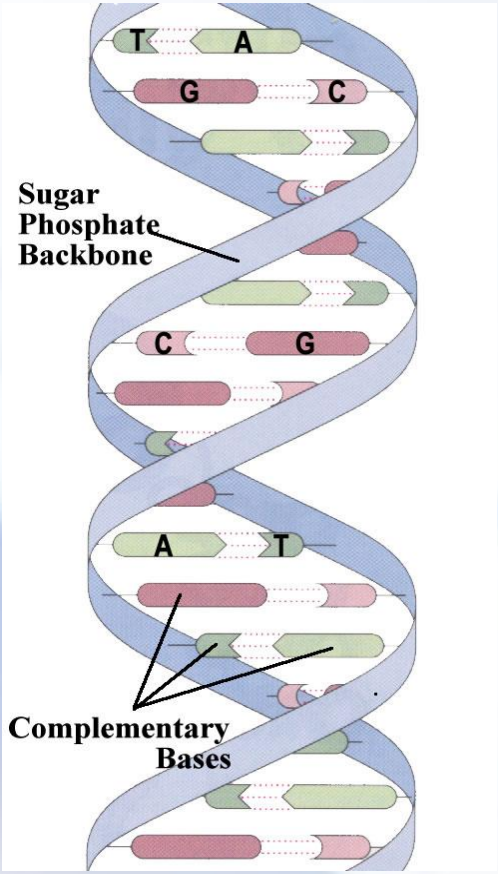
- Genes are encoded in strands of DNA called chromosomes
- In most cells, there are two copies of each chromosome (diploidy)
- The complete genetic material in an individual's genotype is called the Genome



Introduction to DNA

- DNA stands for Deoxyribonucleic Acid
- DNA represents the genetic blueprint of living creatures
- DNA contains “instructions” for assembling cells
- Every cell in human body has a complete set of DNA
- DNA is unique for each individual

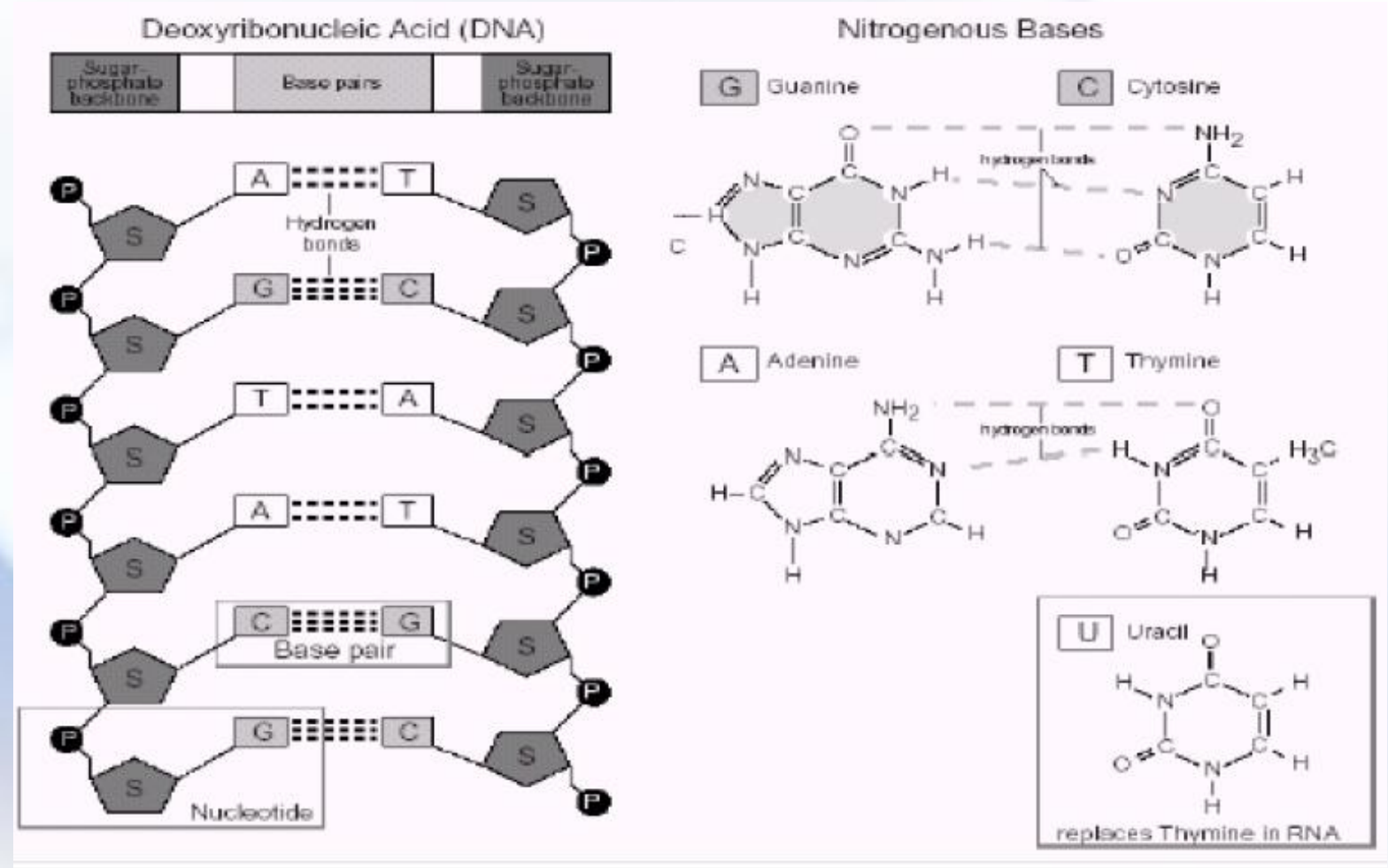
Double Helix



- "*Sides*"
  Sugar-phosphate backbones
- "*ladders*"
  complementary base pairs
  Adenine & Thymine
  Guanine & Cytosine
- Two strands are held together by weak hydrogen bonds between the complementary base pairs

Source: "Human Physiology: From Cells to System 4th Ed.", L. Sherwood, Brooks/Cole, 2001, C-3

DNA Structure



Major Agents of Genetic Change in Individuals: Recombination

- **Produce an offspring genome by taking genes from more than one parent genomes, e.g. , during crossover the chromosome pairs link up and swap parts of themselves:**

Before **After**



Image from Ben Paechter: Evolutionary Computing – A Practical Introduction

Major Agents of Genetic Change in Individuals: Mutation

- Occasionally some of the genetic material changes very slightly during this process
- This means that the child might have genetic material information not inherited from either parent
- This is most likely to be catastrophic

Theory of Evolution

- **Mutation, Recombination → New genetic material or new combinations.**
- **More reproduction leads to more of the "new improved" genetic**
  - "Good" sets of genes get reproduced more
  - "Bad" sets of genes get reproduce less
- **Organisms as a whole get better and better at surviving in their environment.**

Evolution as Search

- **Stringent unending competitive selection coupled with broad range of potential diversity leads to optimized behaviors**

  - Darwin's "Organs of Extreme Perfection"

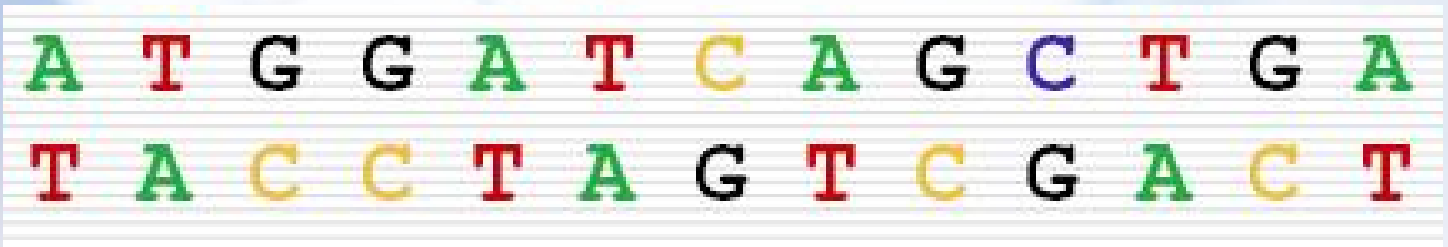
- **Therefore, adoption of the evolutionary paradigm to computation and other problems can help us find optimal solutions:**

  Evolutionary Computation

Part Ⅱ：DNA Computing

DNA Memory

- A DNA string can be viewed as a memory resource to save info:
  - 4 types of units (A,C,G,T) > numbers in base 4
  - Complementary units: A-T,C-G
  - Double-stranded strings



Can DNA Compute?

- DNA itself does not carry out any computation. It rather acts as a massive memory.
- BUT, the way complementary bases react with each other can be used to compute things.
- Proposed by Adelman in 1994 to solve Hamiltonian Path Problem with DNA.

Uniqueness of DNA

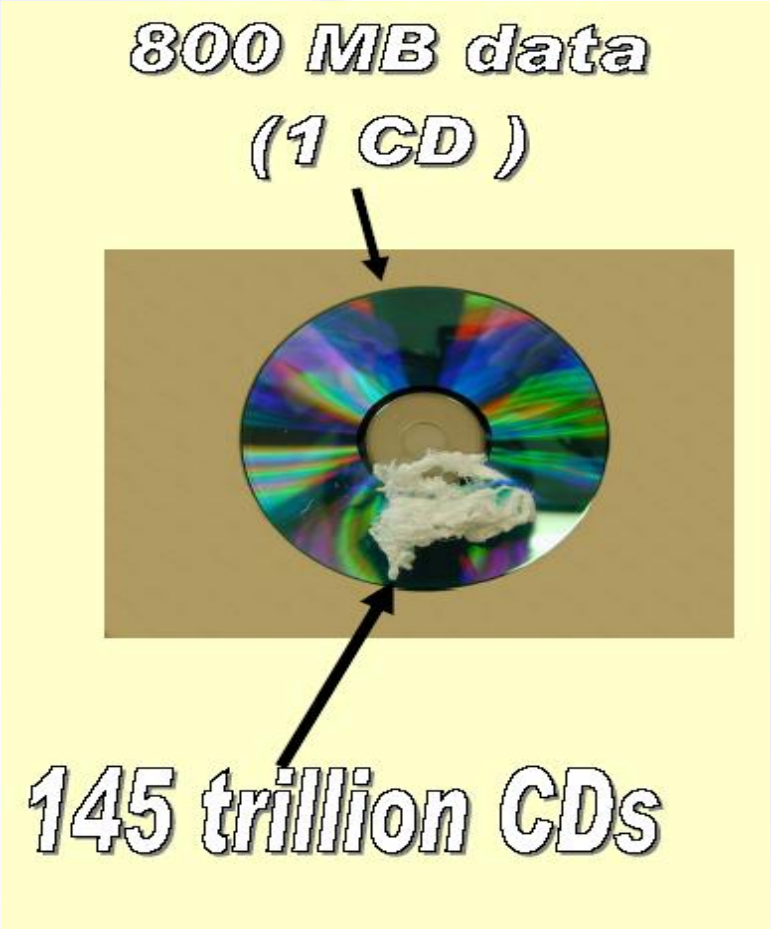
- Extremely dense information storage.
- Enormous parallelism.
- Extraordinary energy efficiency.

Dense Information Storage

✓This image shows 1 gram of DNA on a CD. The CD can hold 800 MB of data.

✓The 1 gram of DNA can hold about $1x10^{14}$ MB of data.

✓The number of CDs required to hold this amount of information, lined up edge to edge, would circle the Earth 375 times, and would take 163,000 centuries to listen to.



How enormous is the parallelism?

- A test tube of DNA can contain trillions of strands. Each operation on a test tube of DNA is carried out on all strands in the tube in parallel !
- Check this out……. We Typically use

300,000,000,000,000

molecules at a time.

How extraordinary is the energy efficiency?

- Adleman figured his computer was running

  2×10^{19} operations per joule.

1 Joule
20,000,000,000,000,000,000 Operations

Molecular revolution

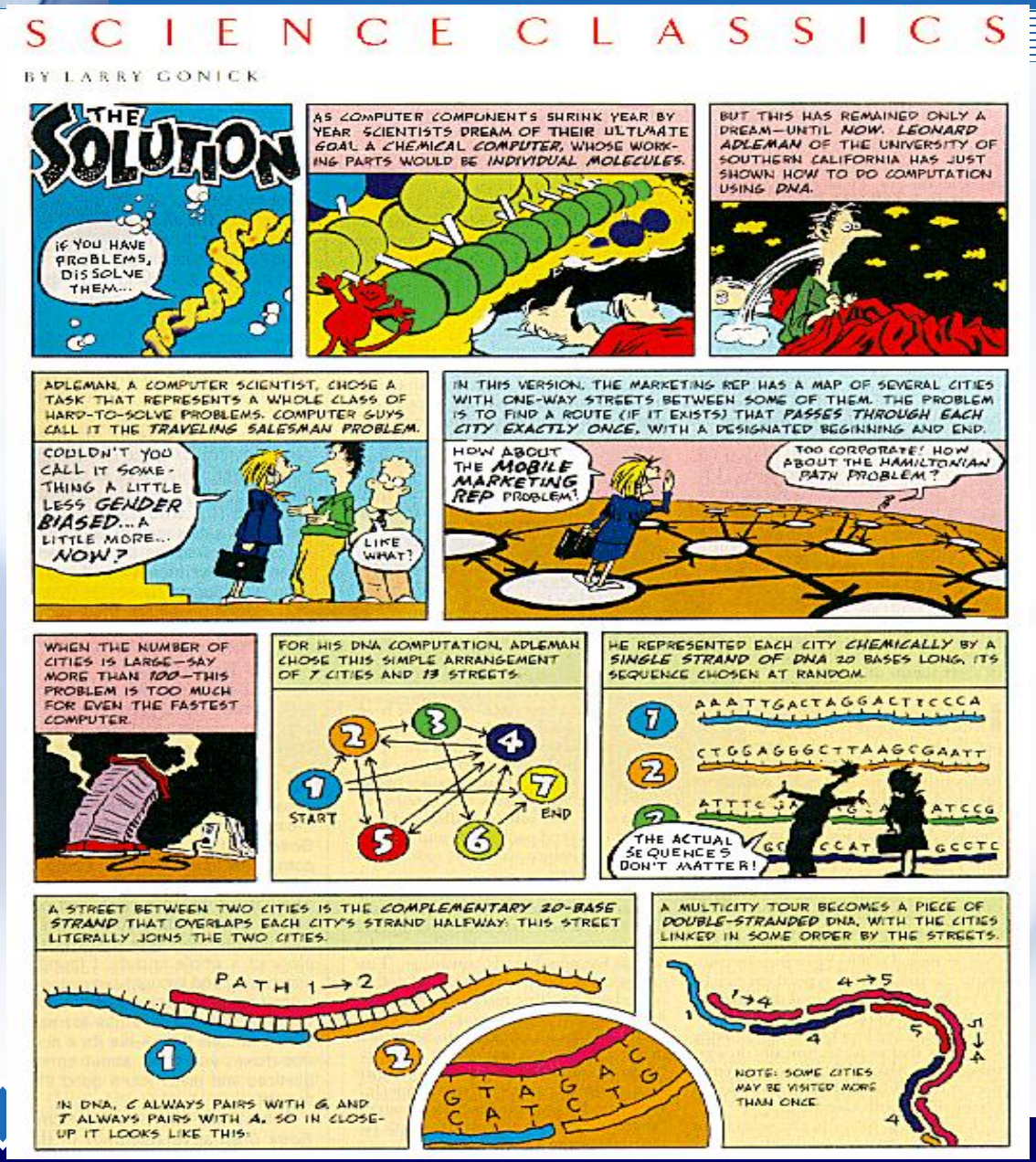
I believe things like DNA computing will eventually lead the way to a "molecular revolution," which ultimately will have a very dramatic effect on the world.

– L.Adleman



[From Adleman's Page]

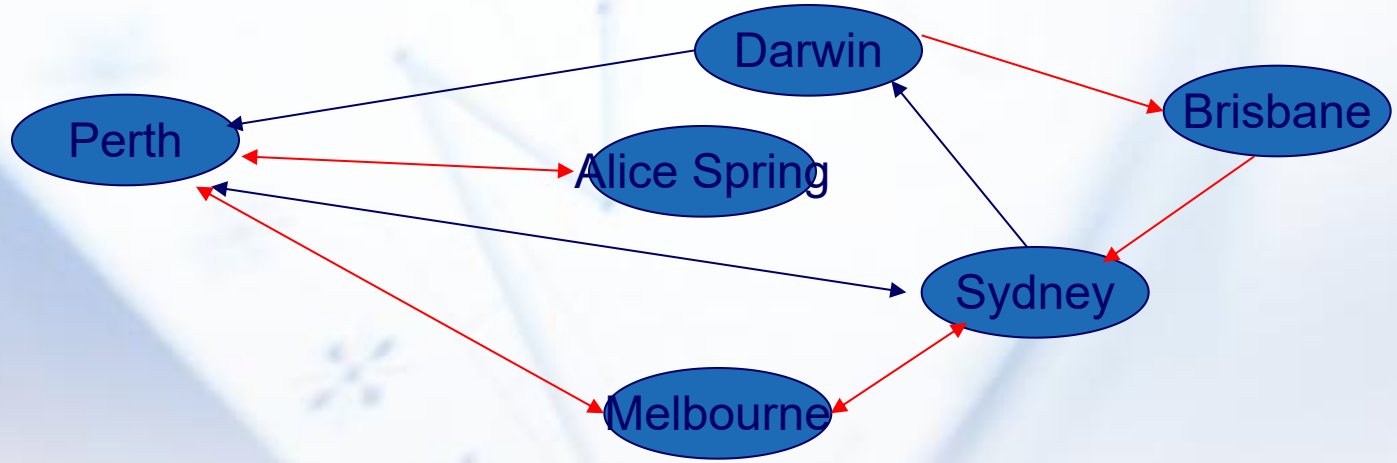
DNA computation comic



Discover magazine published an article in comic strip format about Leonard Adleman's discovery of DNA computation.

Adleman's Experiment (Cont.)

- **Solution by inspection is:**
  Darwin → Brisbane → Sydney → Melbourne → Perth → Alice Spring
- **BUT, there is no deterministic solution to this problem, i.e. we must check all possible combinations.**



Silicon Computer in HPP Problem

- Memory required to store all possible routs = 45 Million GB (20 Vertices)
- Time taken for a 100 MIPS Computer to generate all paths = 2 Years

DNA Computer in HPP Problem

- Finding solution is a matter of few hours
  - No of DNA strands increases exponentially after every iterations > *A test tube of DNA can contain trillions of strands.*
  - Each operation on a test tube of DNA is carried out on all strands in the tube in parallel ! > *Think of 300,000,000,000,000 molecules undergoing parallel processing at a time*

Adleman's Experiment (Cont.)

1. **Encode each city with complementary base - *vertex molecules***

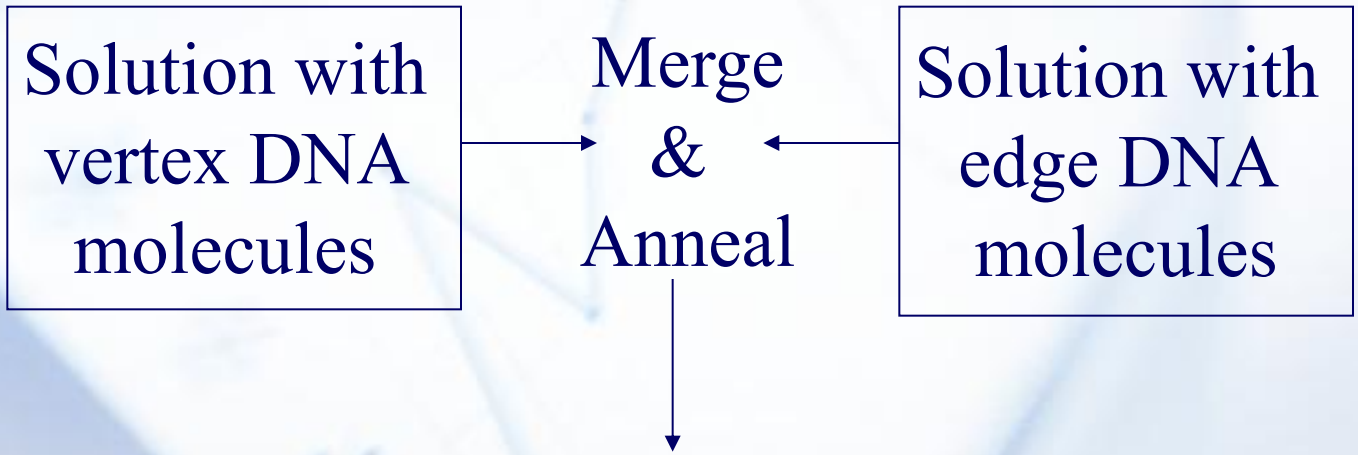
   Sydney - TTAAGG
   Perth - AAAGGG
   Melbourne - GATACT
   Brisbane - CGGTGC
   Alice Spring – CGTCCA
   Darwin - CCGATG

Adleman's Experiment (Cont)

2. **Encode all possible paths using the complementary base – *edge molecules***
   Sydney → Melbourne – AGGGAT (TCCCTA)
   Melbourne → Sydney – ACTTTA (TGAAAT)
   Melbourne → Perth – ACTGGG (TGACCC)
   etc…

Adleman's Experiment (Cont)

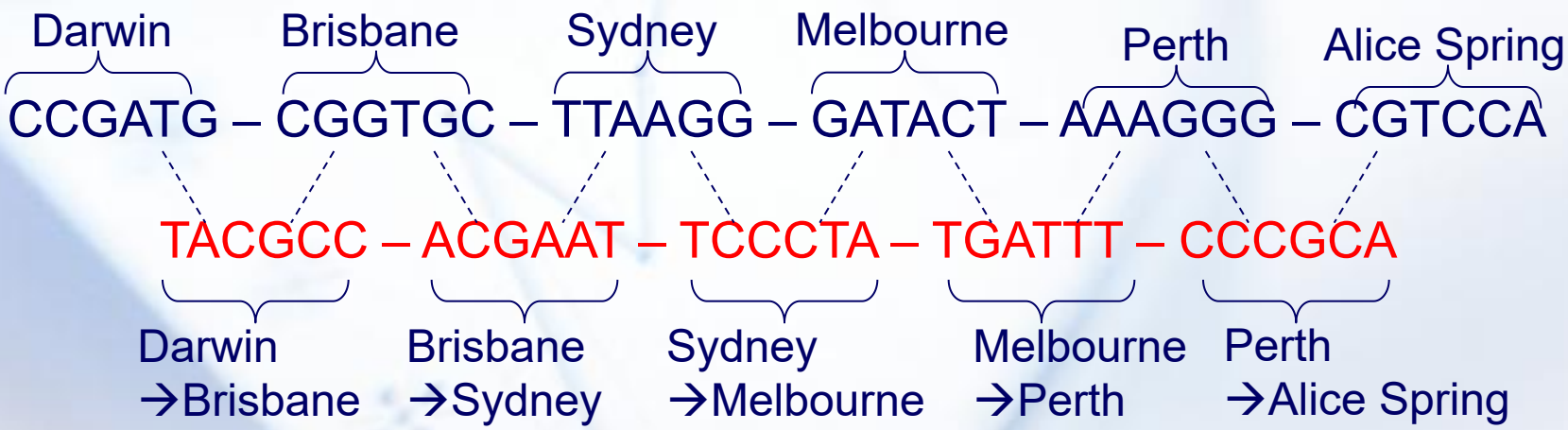
3. **Merge vertex molecules and edge molecules. All complementary base will adhere to each other to form a long chains of DNA molecules**



Adleman's Experiment (Cont)

☆Abracadabra☆

- **The solution is a double helix molecule:**



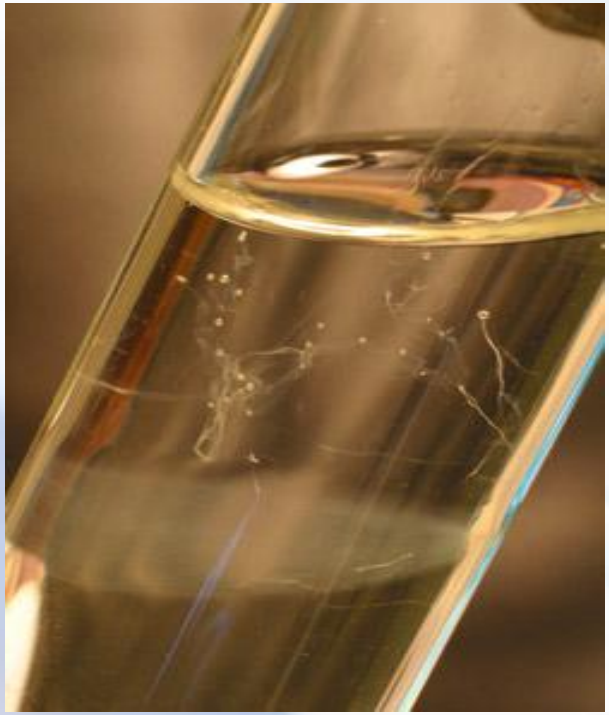
Adleman's Experiment: Operations

- **Melting**
  breaking the weak hydrogen bonds in a double helix to form two DNA strands which are complement to each other
- **Annealing**
  reconnecting the hydrogen bonds between complementary DNA strands

Adleman's Experiment: Operations

- **Merging**
  mixing two test tubes with many DNA molecules
- **Amplification**
  DNA replication to make many copies of the original DNA molecules
- **Selection**
  elimination of errors (e.g. mutations) and selection of correct DNA molecules

Adleman's Experiment: Process



Formation of DNA strands.



Precipitation of more DNA strands in alcohol

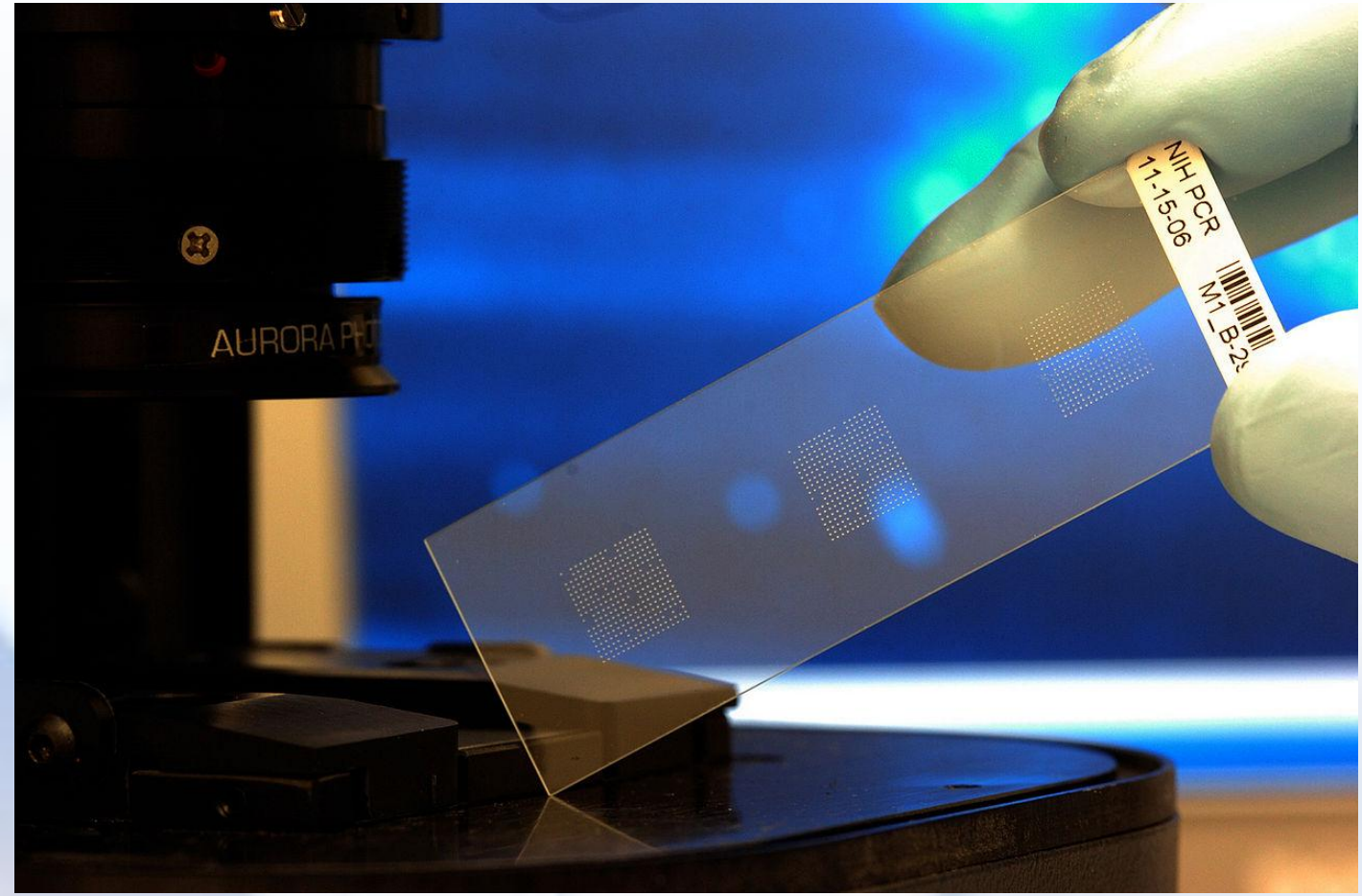


Spooling the DNA with a metal hook or similar device

DNA Computing vs. Conventional Computing

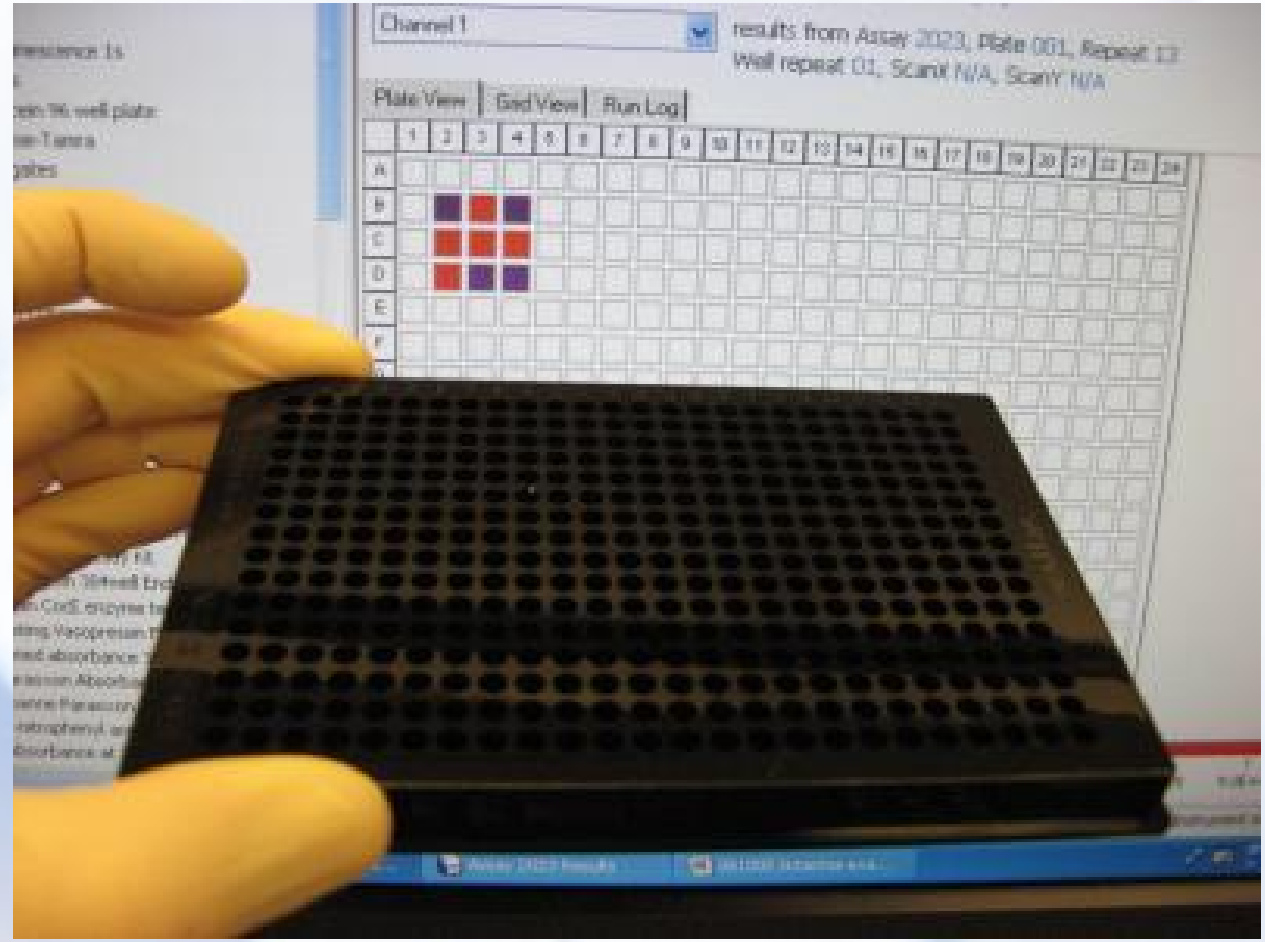
DNA-based computers	Microchip-based computers
slow at individual operations	fast at individual operations
can do billions of operations simultaneously	can do substantially fewer operations simultaneously
can provide huge memory in small space	smaller memory
setting up a problem may involve considerable preparations	setting up only requires keyboard input
DNA is sensitive to chemical deterioration	electronic data are vulnerable but can be backed up easily

Cutting edge technologies: DNA Chip



https://calvinchimes.org/2014/11/12/dna-computer-chip-technology/

Cutting edge technologies: DNA Computer



https://www.eurekalert.org/multimedia/pub/2143.php

Part Ⅲ: What is Evolutionary Computation?

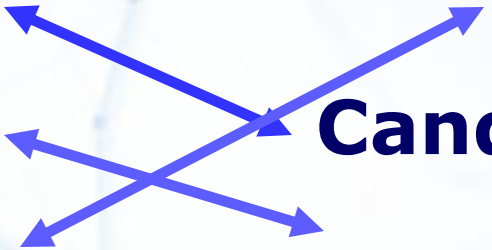
3.1 The Metaphor

EVOLUTION

Individual

Fitness

Environment



SEARCH

Problem

Candidate Solution

Quality

The Ingredients

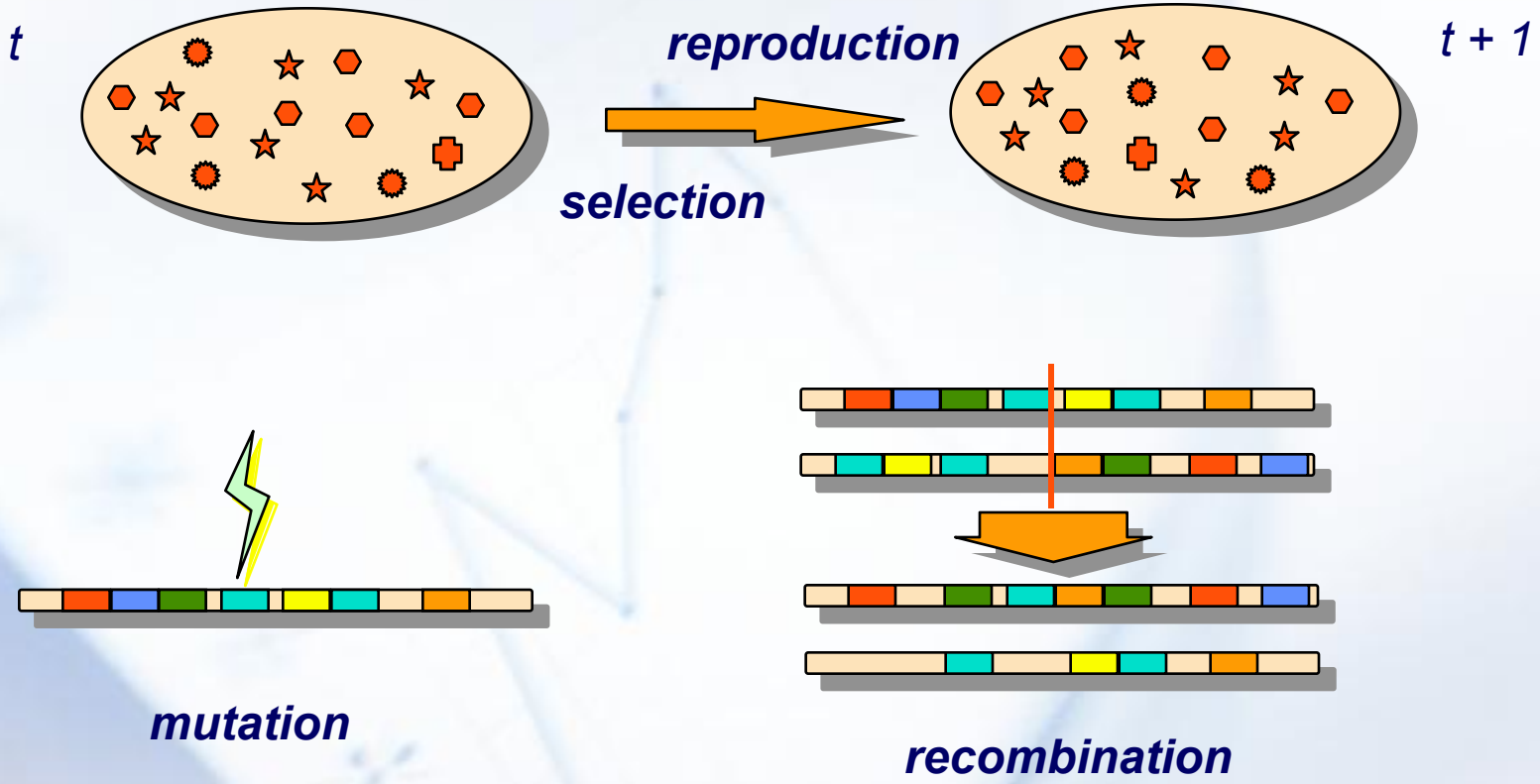


Image from Ida Sprinkhuizen-Kuyper: Introduction to Evolutionary Computation, 2000.

The Evolution Mechanism

- **Increasing diversity by genetic operators**
  - mutation
  - Recombination
- **Decreasing diversity by selection**
  - of parents
  - of survivors

The Evolutionary Cycle

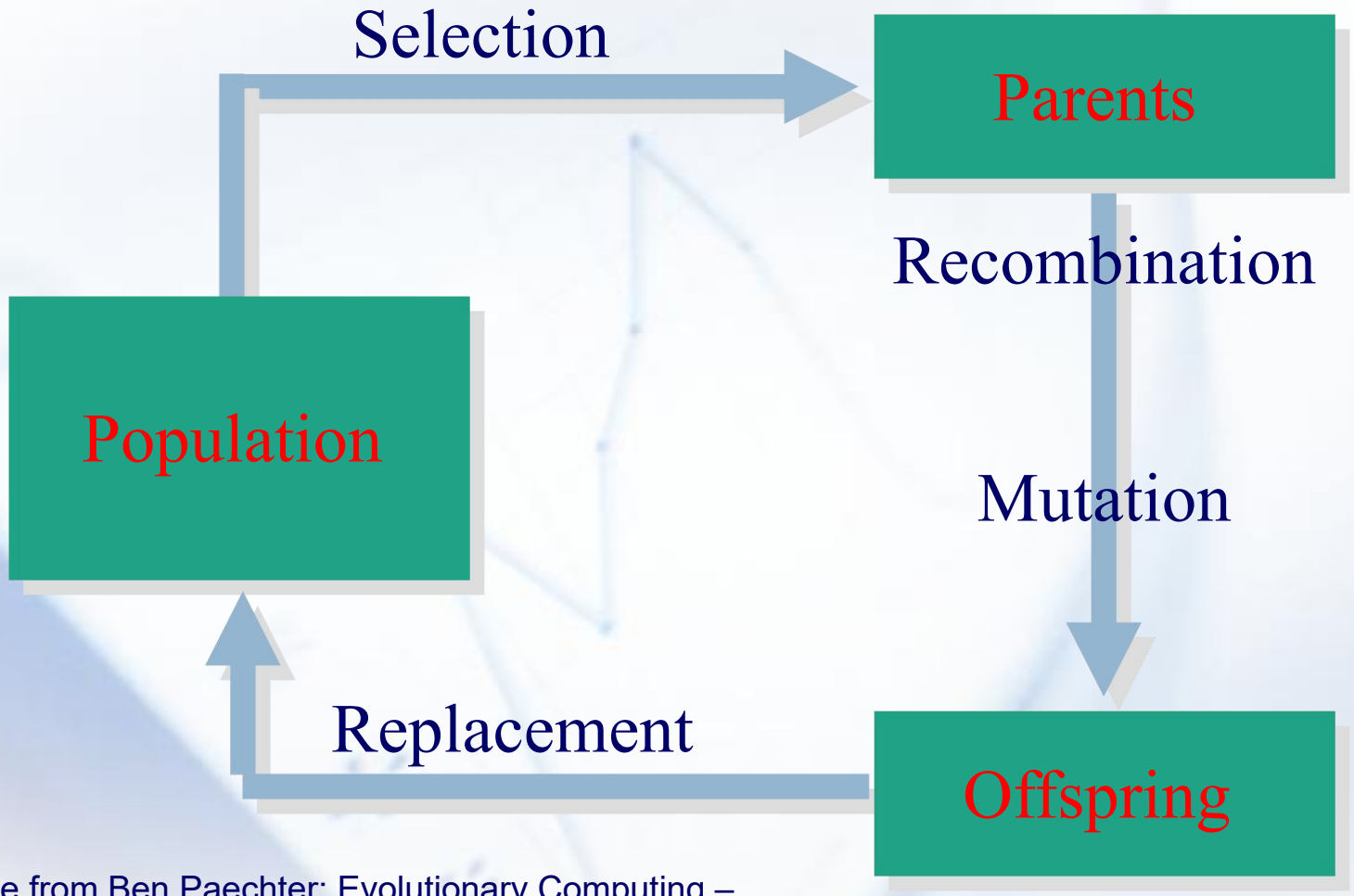


Image from Ben Paechter: Evolutionary Computing – A Practical Introduction

Desirable Performance of EC

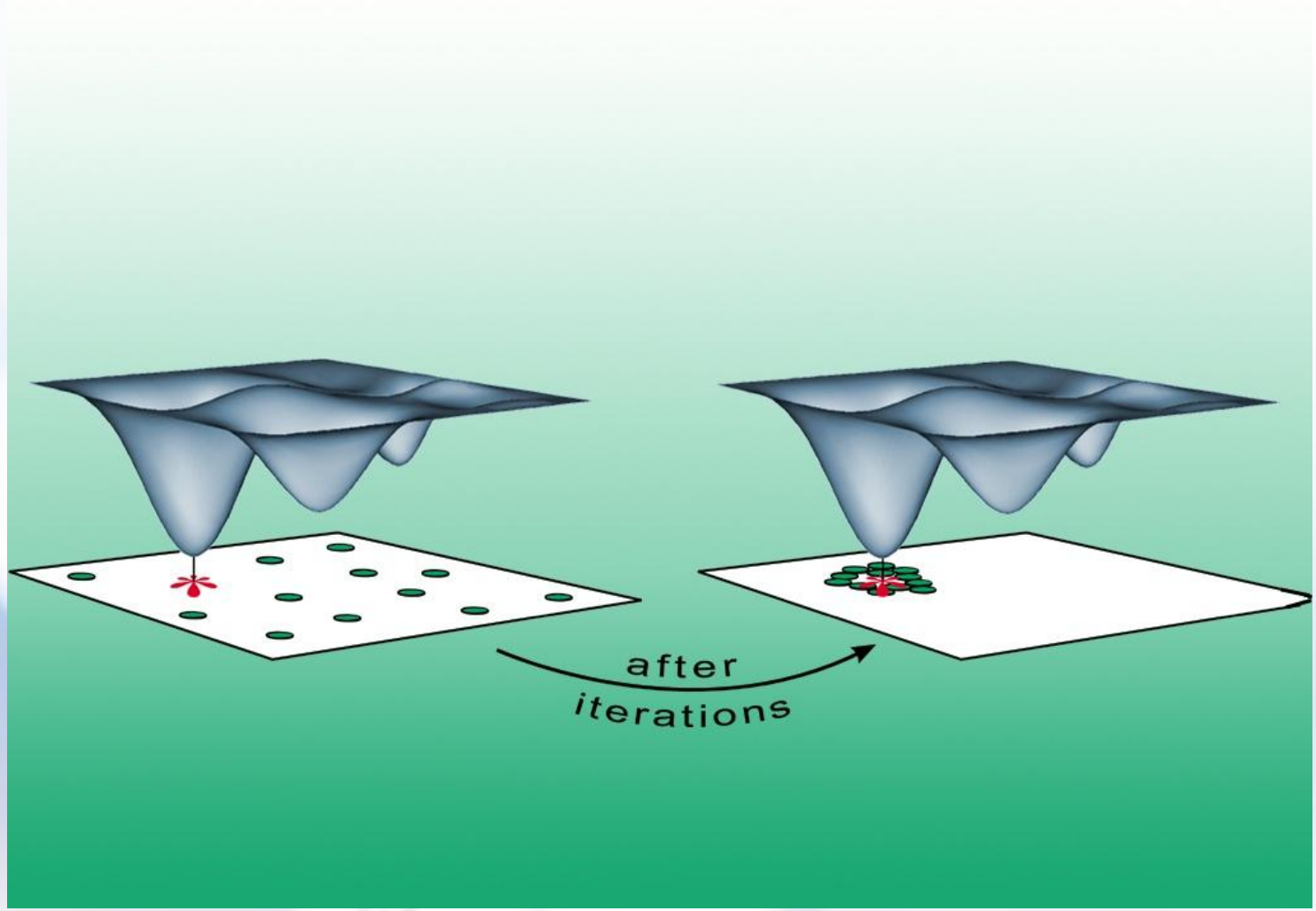


Image from: Introduction to Stochastic Search and Optimization (ISSO) by J. C. Spall

Pseudo-code for typical Evolutionary Algorithm (EA)

```
BEGIN
  INITIALISE population with random candidate solutions;
  EVALUATE each candidate;
  REPEAT UNTIL ( TERMINATION CONDITION is satisfied ) DO
    1 SELECT parents;
    2 RECOMBINE pairs of parents;
    3 MUTATE the resulting offspring;
    4 EVALUATE new candidates;
    5 SELECT individuals for the next generation;
  OD
END
```

Image from: A.E. Eiben and J.E. Smith, Introduction to Evolutionary Computing

3.2 Five factors

- Representation
  - Genotypic vs. Phenotypic
- Fitness Evaluation
- Genetic Operations
  - Recombination
  - Mutation
- Selection Strategies
  - Parent Selection
  - Survivor Selection
- Initialization\Termination Schemes

（1）Representations

- **Genotypic representations:**
  - Encoding : phenotype=> genotype (not necessarily one to one)
  - Decoding : genotype=> phenotype (must be one to one)

- **Phenotypic representations:**
  - problem-specific encodings

（2）Evaluation (Fitness) Function

- **Represents the requirements that the population should adapt to**
- **i.e., *quality* function or *objective* function**
- **Assigns a single real-valued fitness to each phenotype which forms the basis for selection**
  - So the more discrimination (different values) the better

（3）Genetic Operators

- **Role is to generate new candidate solutions**
- **Usually divided into two types according to the number of parents:**
  - = 1 : mutation operators
  - >1 : Recombination operators
  - = 2: crossover operators
- **There has been much debate about relative importance of recombination and mutation**
  - Nowadays most EAs use both

Discuss: Crossover OR mutation?

- Only crossover can combine information from two parents
- Only mutation can introduce new information (alleles)

  Crossover does not change the allele frequencies of the population
- To hit the optimum you often need a 'lucky' mutation

（4）Parent Selection Mechanism

- **Assigns variable probabilities of individuals acting as parents depending on their fitnesses**
  - high quality solutions more likely to become parents than low quality, but not guaranteed
  - even worst in current population usually has non-zero probability of becoming a parent
- **This *stochastic* nature can aid escape from local optima.**

Survivor Selection

- **a.k.a. *replacement***
- **A way of going from (parents + offspring) to next generation**
- **Two ways**
  - Fitness based : e.g., rank parents+offspring and take best
  - Age based: make as many offspring as parents and delete all parents

Exploitation/Exploration Balance

- **Selection: exploitation**
  - reduce scope of search, try to improve the best-so-far individuals
  - Too much exploitation = local search only … convergence to a local optimum
- **Genetic Operators: exploration**
  - expand scope of search, sample unknown regions
  - Too much exploration = random search, no convergence
- **Key issue: appropriate balance**
  - e.g., strong selection + high mutation rates
  - e.g, weak selection + low mutation rates

（5）Initialisation / Termination

- **Initialisation usually done at random,**
- **Termination condition checked every generation**
  - Reaching some (known/hoped for) fitness
  - Reaching some maximum allowed number of generations
  - Reaching some minimum level of diversity
  - Reaching some specified number of generations without fitness improvement

3.3 How to build a EA?

Step1. Design a representation

Step2. Design a way of mapping a genotype to a phenotype (not necessarily)

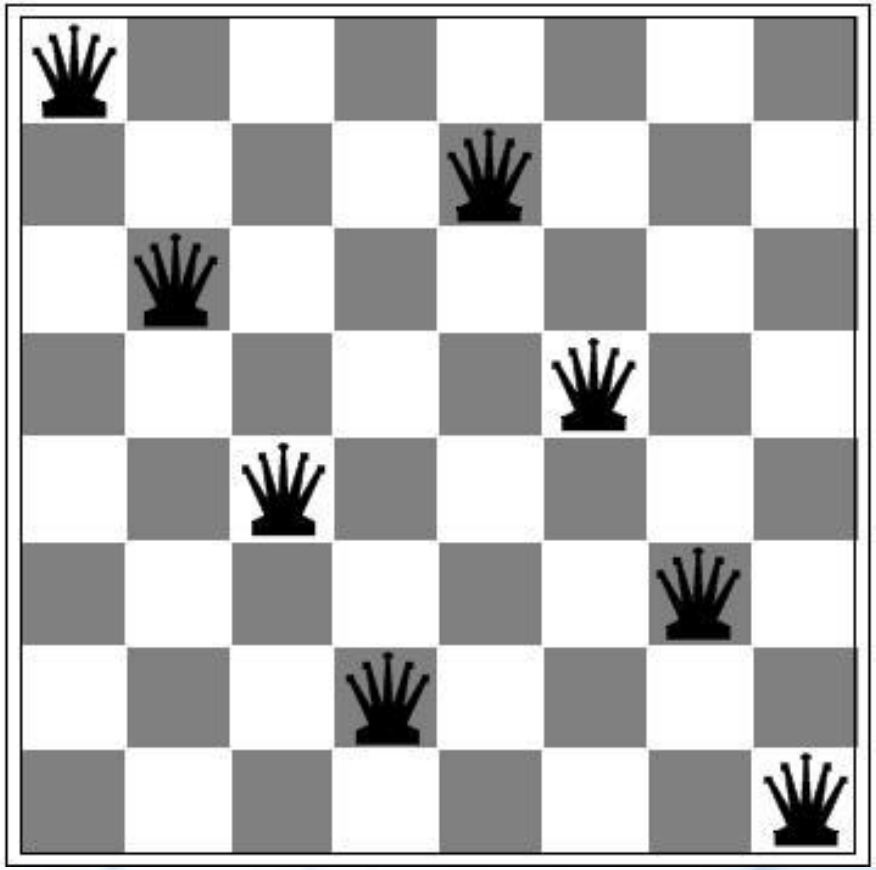
Step3. Design a fitness function

Step4. Design suitable genetic operators: mutation and/or recombination

Step5. Decide how to select parents and survivors

Step6. Decide how to initialise a population and when to stop the algorithm

Example: the 8 queens problem [After A.E. Eiben and J.E. Smith, Introduction to Evolutionary Computing]

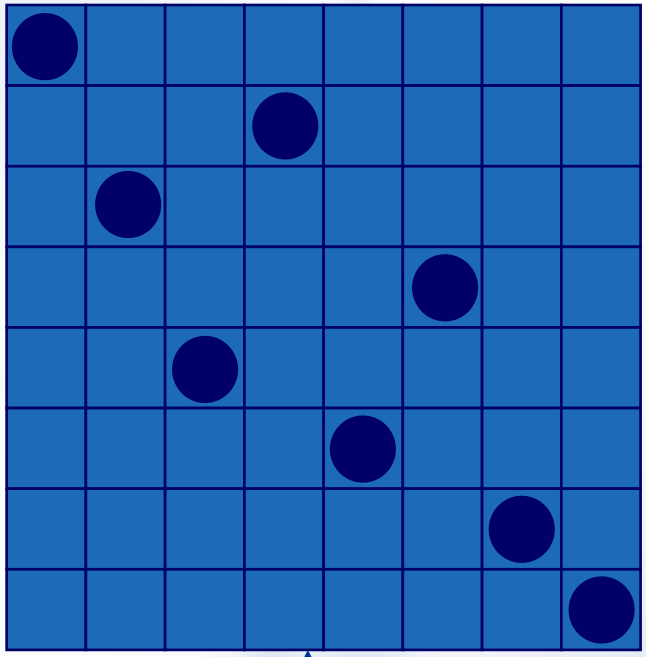


Place 8 queens on an 8x8 chessboard in such a way that they cannot check each other

8 queens problem: representation

Phenotype:
a board configuration

Genotype:
a permutation of
the numbers 1 - 8



Obvious mapping

1	3	5	2	6	4	7	8

8 Queens Problem: Fitness evaluation

- Penalty of one queen:
  the number of queens she can check.
- Penalty of a configuration:
  the sum of the penalties of all queens.
- Note: penalty is to be minimized
- Fitness of a configuration:
  inverse penalty to be maximized

8 queens problem: Genetic Operators

- Mutation:

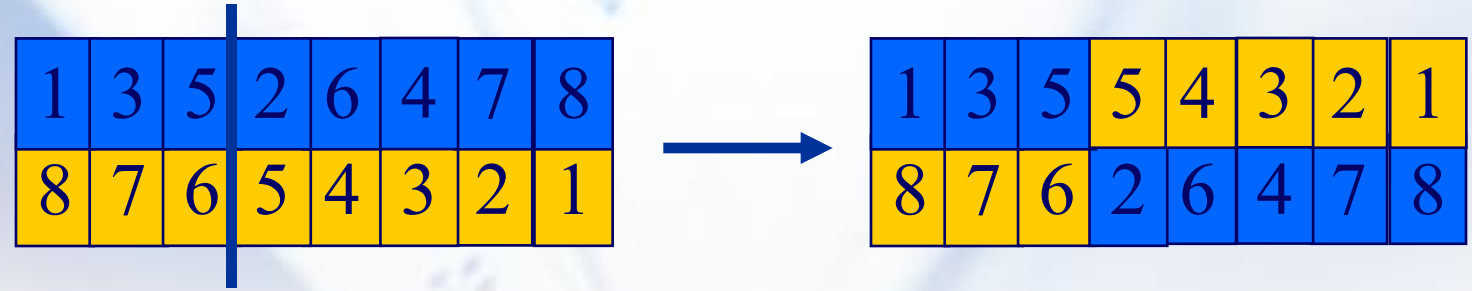
  swapping values of two randomly chosen positions

  (probability: 80%)



- Recombination:

  crossover (probability: 100%)



8 queens problem: Selection

- **Parent selection:**
  - Pick 5 parents and take best two to undergo crossover
- **Survivor selection (replacement)**
  - When inserting a new child into the population, choose an existing member to replace by:
    - sorting the whole population by decreasing fitness
    - enumerating this list from high to low
    - replacing the first with a fitness lower than the given child

8 queens problem: Initialization\Termination

- **Initialization:**
  - Random
- **Termination:**
  - Solution or Maximum Times (10,000) of Fitness Evaluation

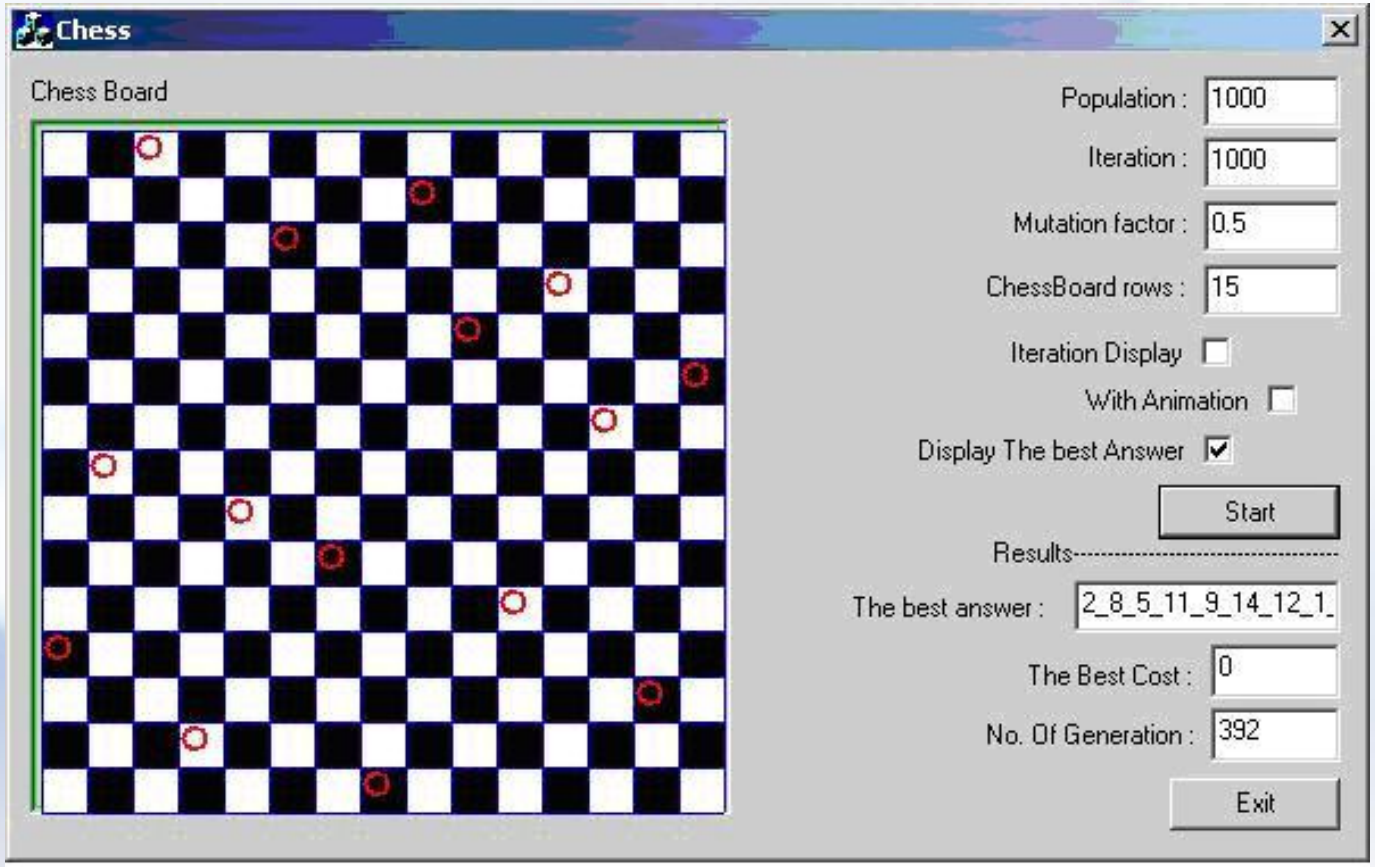
8 Queens Problem: summary

Representation	Permutations
Recombination	"Cut-and-crossfill" crossover
Recombination probability	100%
Mutation	Swap
Mutation probability	80%
Parent selection	Best 2 out of random 5
Survival selection	Replace worst
Population size	100
Number of Offspring	2
Initialisation	Random
Termination condition	Solution or 10,000 fitness evaluation

Note that this is ***only one possible***
set of choices of operators and parameters

8 Queens Solution by EA: A Demo

from http://www.codeproject.com/KB/recipes/8queenssolution.aspx



Advantages of EC

- Widely applicable to continuous, discrete, mixed optimization problems.
- no *a priori* assumptions about convexity, continuity, differentiability, etc.
- relatively insensitive to noise
- easy to parallelize

Disadvantages of EC

- No guarantee for optimal solution within finite time
- Weak theoretical basis
- Need parameter tuning

No Free Lunch Theorem

Part Ⅳ : Evolutionary Algorithms

Main Streams of EC

- **Genetic Algorithms (GA)**
  - J. Holland 1962 (Ann Arbor, MI)
- **Evolutionary Programming (EP)**
  - L. Fogel 1962 (San Diego, CA)
- **Evolution Strategies (ES)**
  - I. Rechenberg & H.-P. Schwefel 1965 (Berlin)
- **Genetic Programming (GP)**
  - J. Koza 1989 (Palo Alto, CA)

Similarities are more important than the differences

Technical Summary

	GA	EP	ES	GP
Represent-ation	Genotype	Phenotype	Phenotype	Genotype
Mutation	√	√	√	√
Recombin-ation	√	×	√	√
Parent Selection	Probabilist-ic	Determini-stic	Probabilist-ic	Probabilist-ic
Survivor Selection	Extinctive, Determini-stic	Combined, Probabilist-ic	Combined, Determini-stic	Extinctive, Determini-stic

4.1 Genetic algorithms

- **Holland's original GA is now known as the simple genetic algorithm (SGA)**
  - still often used as benchmark for novel GAs

- **Other GAs use different:**
  - Representations
  - Mutations
  - Crossovers
  - Selection mechanisms

Representation

- **The genome is used to represent candidate solutions**
  - Fixed length Bitstrings (SGA)
    - Holland
    - Traditional
    - Convergence theorems exist
  - Real-valued genomes
    - Artificial evolution
    - Difficult to prove convergence

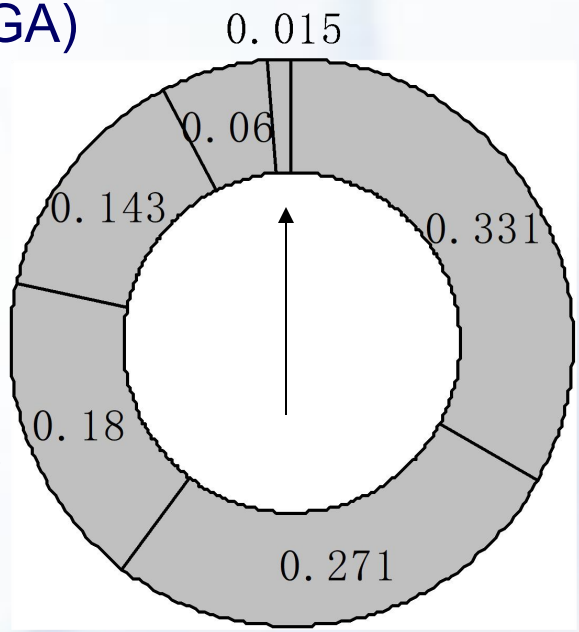
GA operators: Selection

- **Greedy:**
  - the fittest solutions are selected
- **Probabilistic:**
  - Chances proportional to fitness
  - E.g., roulette wheel technique (SGA)

$$\Pr(h_i) = \frac{Fitness(h_i)}{\sum_{j=1}^{p} Fitness(h_j)}$$

Example Fitness:

(2200, 1800, 1200, 950, 400, 100)



GA operators: crossover

- **For given parents, crossover occurs with specified probability ≤ 1(note: P_c typically in range (0.6, 0.9))**
  - Else, parents placed directly into next generation (clones)
- **E.g.,**

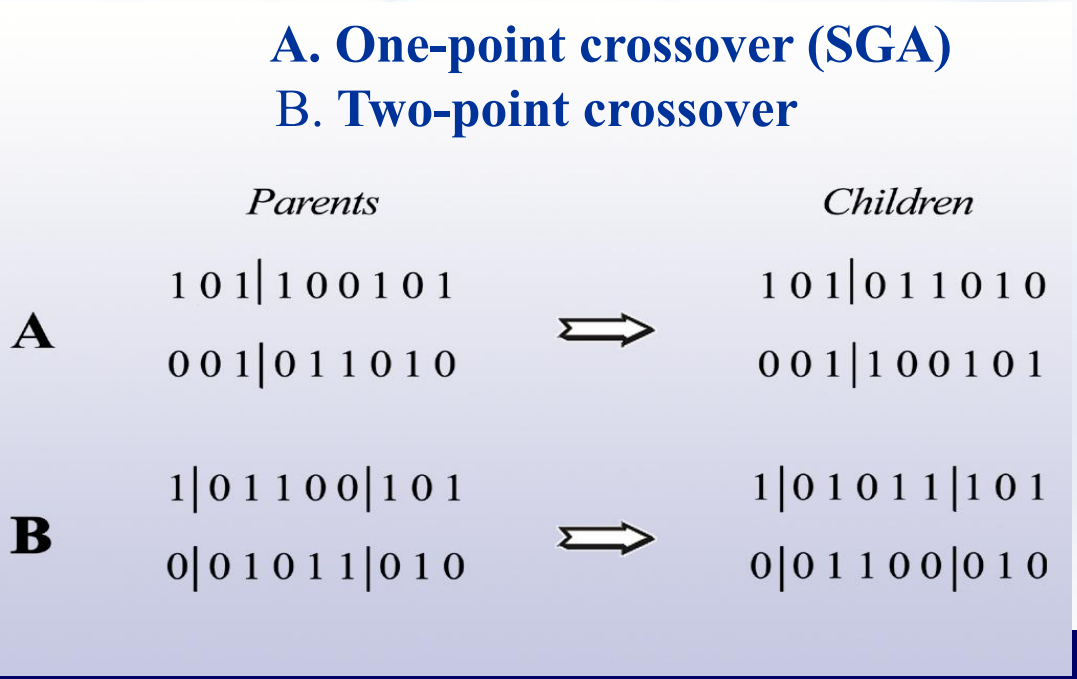
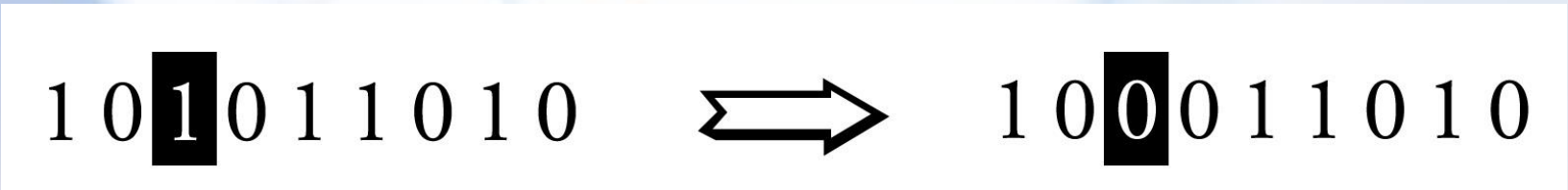


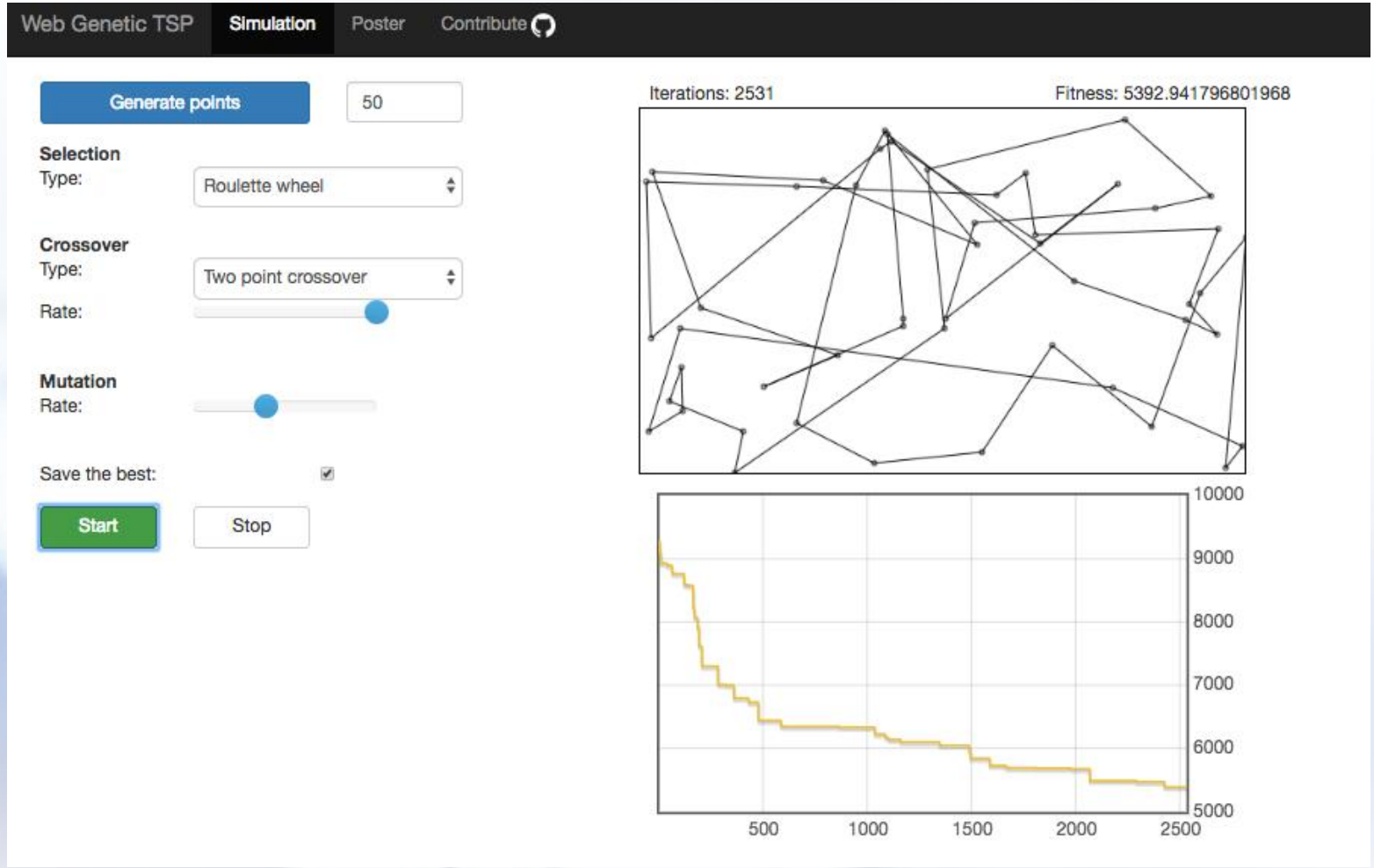
Image from: Introduction to Stochastic Search and Optimization (ISSO) by J. C. Spall

GA operators: mutation

- **Alter each gene independently with a probability p_m**
- **p_m is called the mutation rate**
  - Typically between 1/pop_size and 1/ chromosome_length
- **E.g.(SGA),**



GA Demo: TSP Solver



https://courses.cs.ut.ee/demos/web-ga-tsp-master/

4.2 Genetic Programming

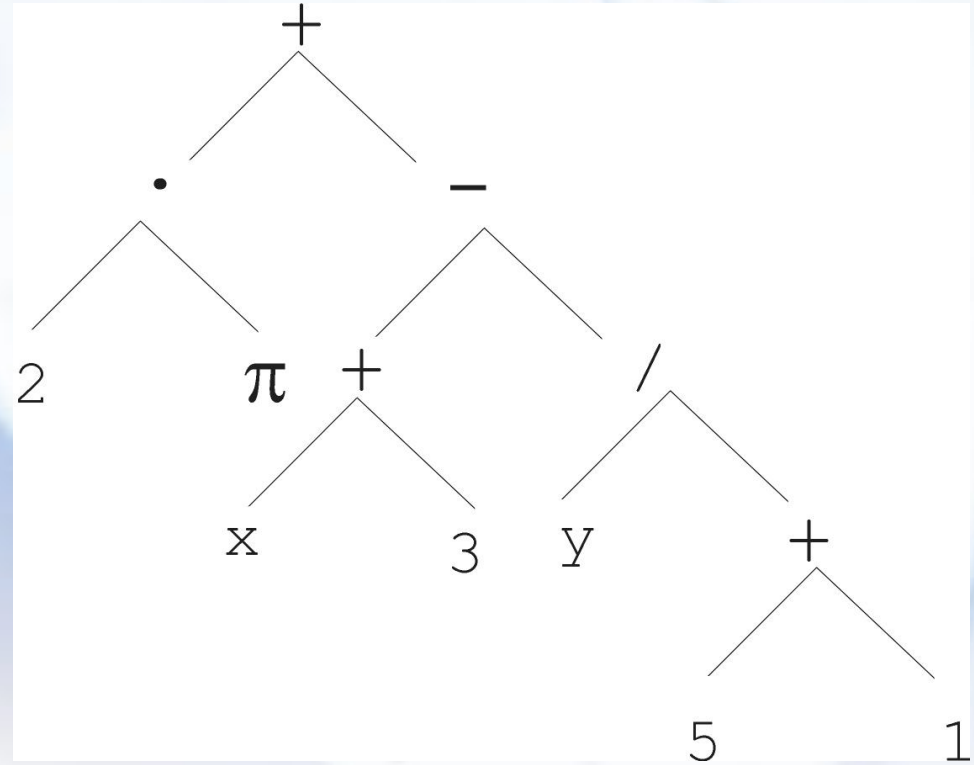
Representation	Tree structures
Recombination	Exchange of subtrees
Mutation	Random change in trees
Parent selection	Fitness proportional
Survivor selection	Generational replacement

Tree based representation

- **Trees are a universal form, e.g. consider**
- **Arithmetic formula** $2 \cdot \pi + \left((x+3) - \frac{y}{5+1} \right)$
- **Logical formula** $(x \wedge \text{true}) \rightarrow ((x \vee y) \vee (z \leftrightarrow (x \wedge y)))$
- **Program**

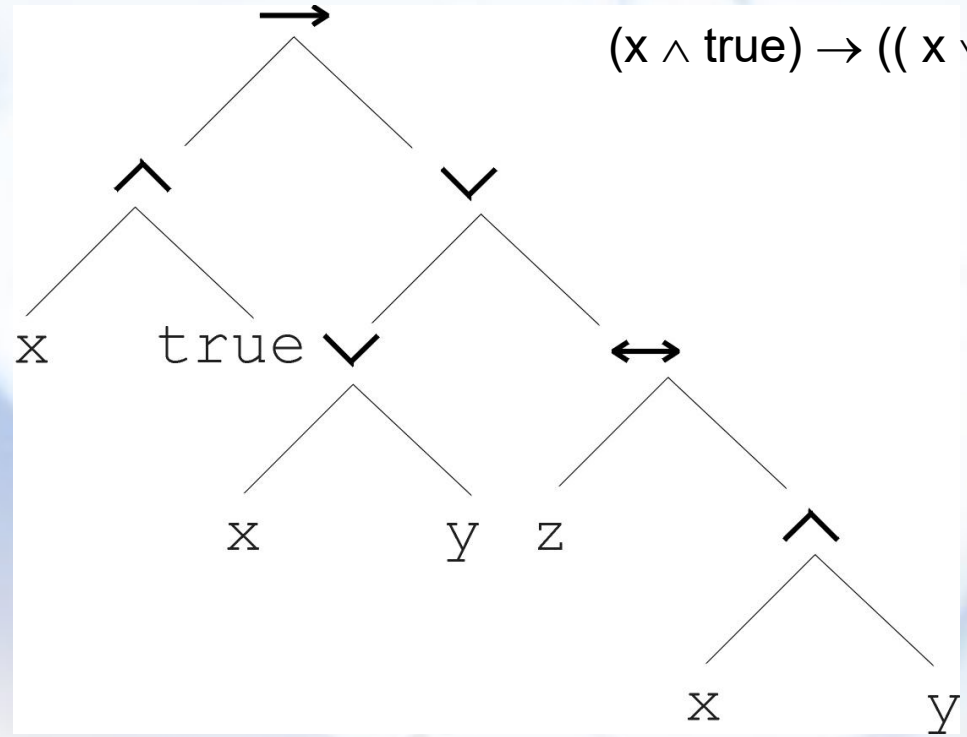
```
i =1;
while (i < 20)
{
        i = i +1
}
```

Tree based representation



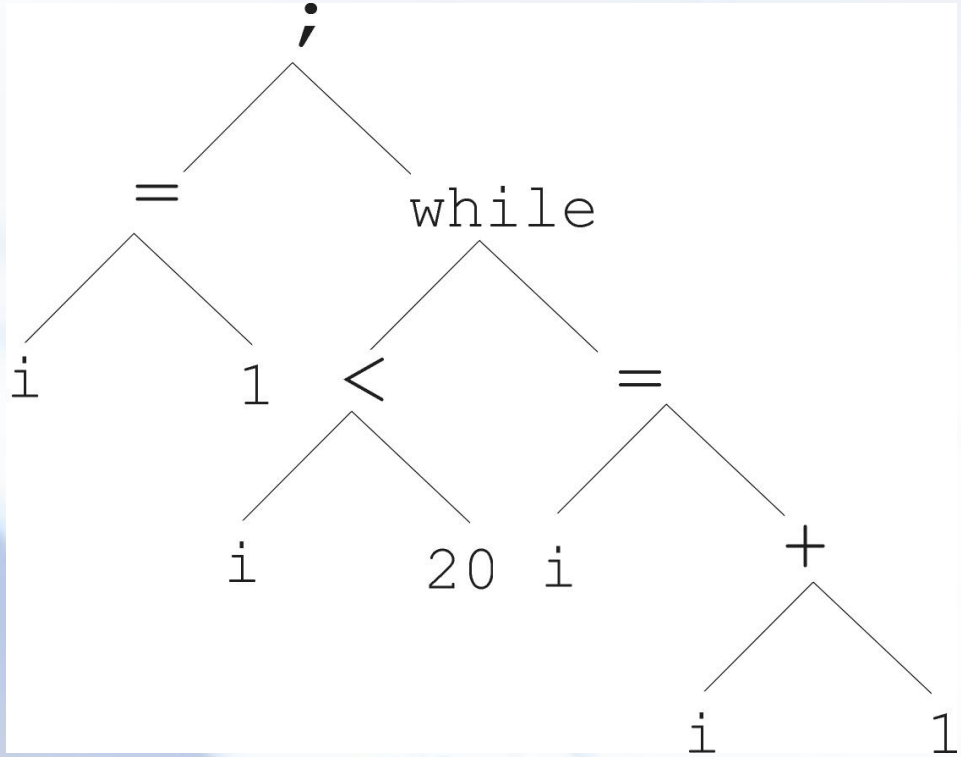
$$2 \cdot \pi + \left((x+3) - \frac{y}{5+1} \right)$$

Tree based representation



$(x \wedge \text{true}) \rightarrow ((x \vee y) \vee (z \leftrightarrow (x \wedge y)))$

Tree based representation



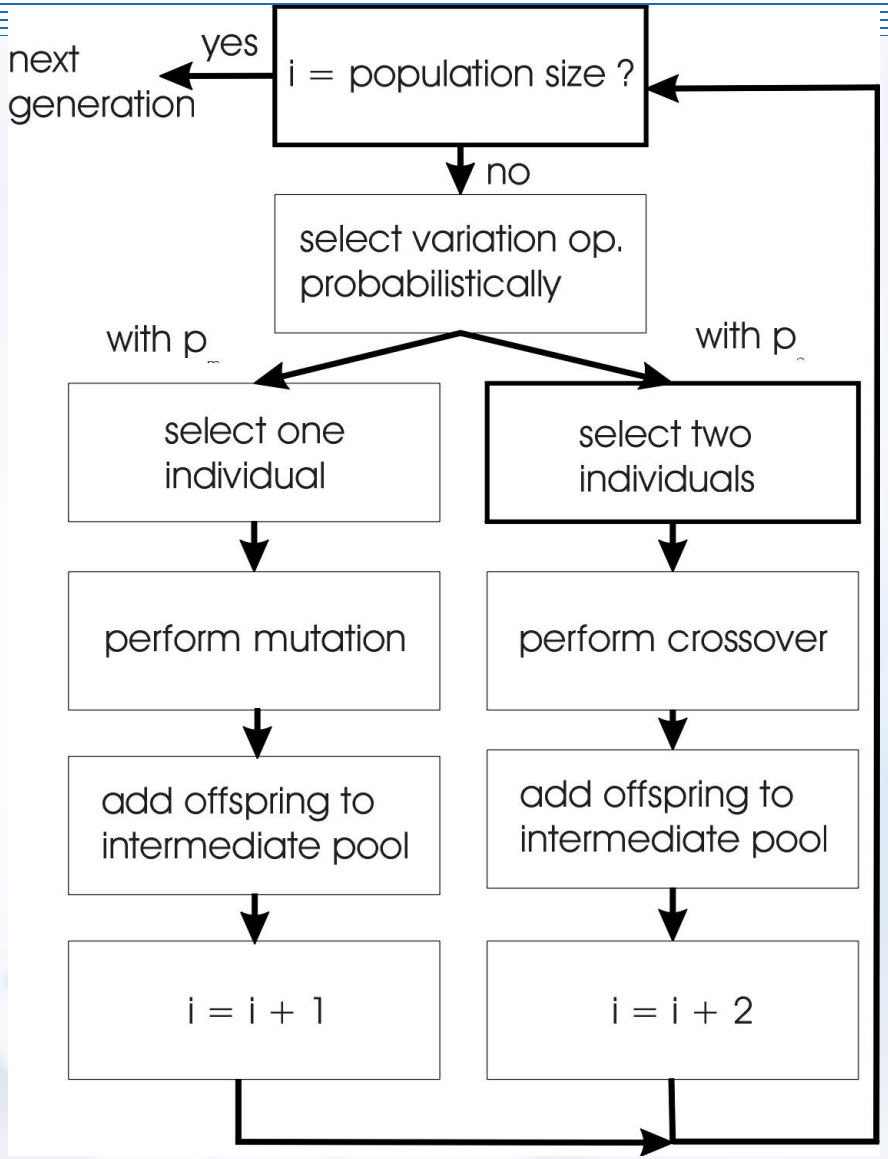

```
i =1;
while (i < 20)
{
        i = i +1
}
```

Tree based representation

- **Symbolic expressions can be defined by**
  - Terminal set T
  - Function set F (with the arities of function symbols)

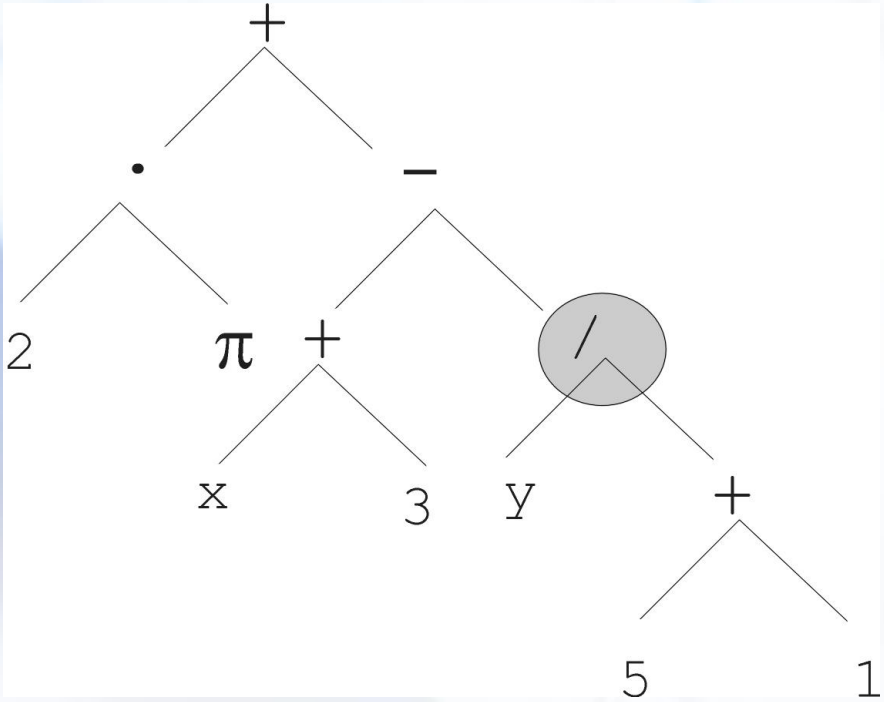
- **Adopting the following general recursive definition:**
  1. Every $t \in T$ is a correct expression
  2. $f(e_1, \ldots, e_n)$ is a correct expression if $f \in F$, arity(f)=n and $e_1, \ldots, e_n$ are correct expressions
  3. There are no other forms of correct expressions

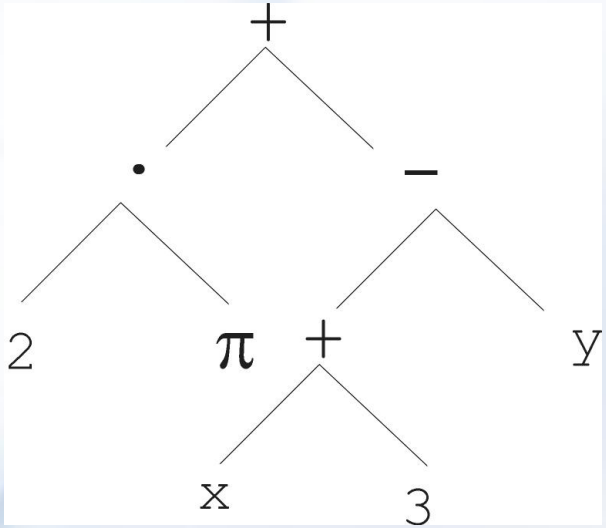
GP flowchart



Mutation

- **Most common mutation: replace randomly chosen subtree by randomly generated tree**



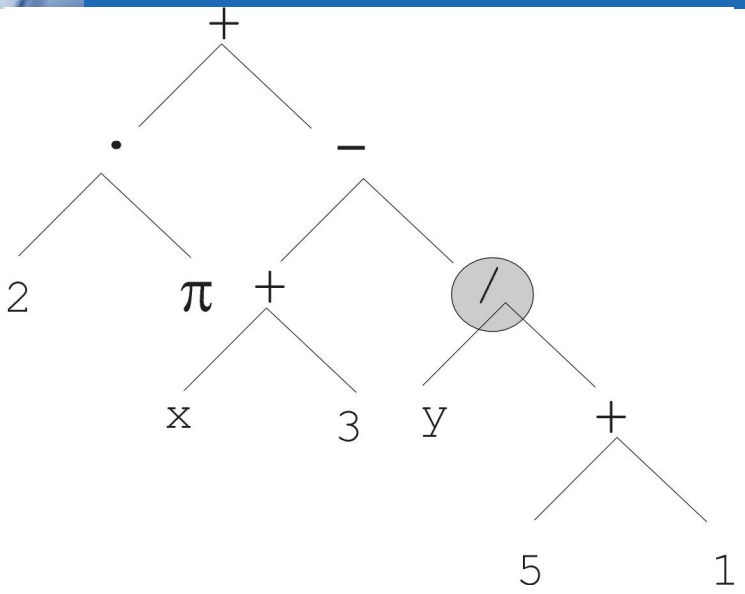


Mutation cont'd

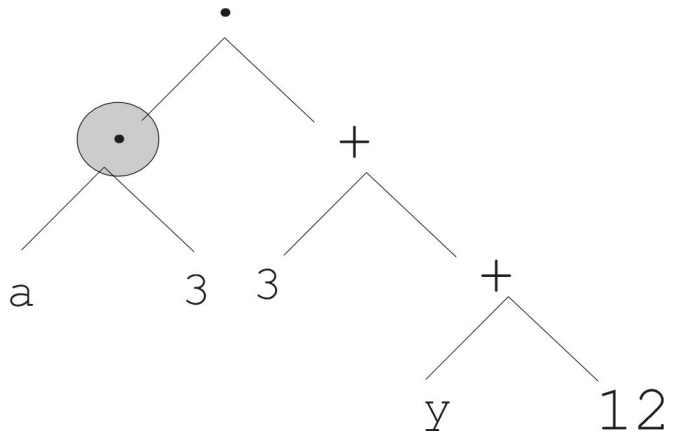
- **Mutation has two parameters:**
  - Probability p_m to choose mutation vs. recombination
  - Probability to chose an internal point as the root of the subtree to be replaced
- **Remarkably p_m is advised to be 0 (Koza'92) or very small, like 0.05 (Banzhaf et al. '98)**
- **The size of the child can exceed the size of the parent**

Recombination

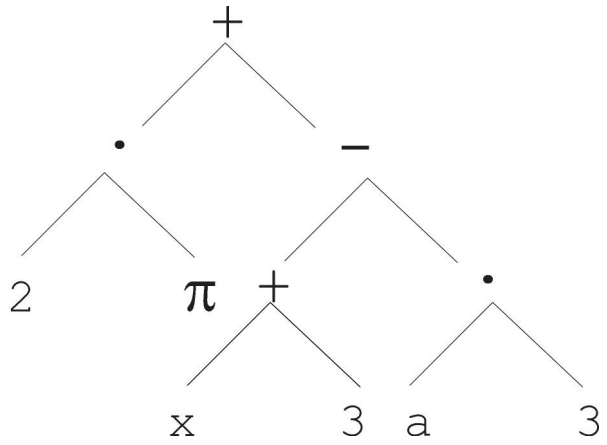
- **Most common recombination: exchange two randomly chosen subtrees among the parents**
- **Recombination has two parameters:**
  - Probability p_c to choose recombination vs. mutation
  - Probability to chose an internal point within each parent as crossover point
- **The size of offspring can exceed that of the parents**



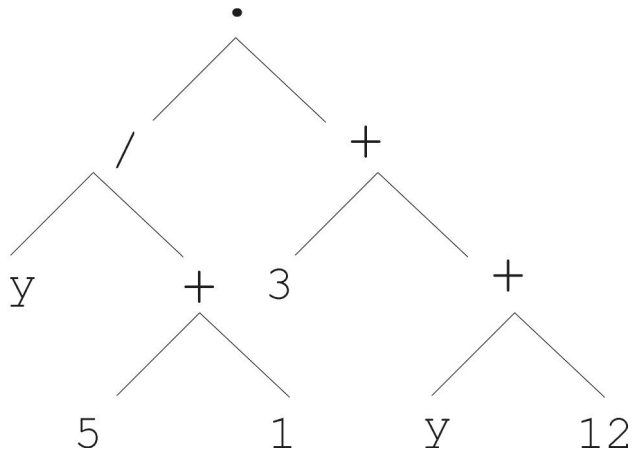
Parent 1



Parent 2



Child 1



Child 2

Selection

- **Parent selection typically fitness proportionate**
- **Over-selection in very large populations**
  - rank population by fitness and divide it into two groups:
  - group 1: best x% of population, group 2 other (100-x)%
  - 80% of selection operations chooses from group 1, 20% from group 2
  - for pop. size = 1000, 2000, 4000, 8000 x = 32%, 16%, 8%, 4%
  - motivation: to increase efficiency, %'s come from rule of thumb
- **Survivor selection:**
  - Typical: generational scheme (thus none)

Initialisation

- **Maximum initial depth of trees D_{max} is set**
- **Full method (each branch has depth = D_{max}):**
  - nodes at depth $d < D_{max}$ randomly chosen from function set F
  - nodes at depth $d = D_{max}$ randomly chosen from terminal set T
- **Grow method (each branch has depth $\leq D_{max}$):**
  - nodes at depth $d < D_{max}$ randomly chosen from $F \cup T$
  - nodes at depth $d = D_{max}$ randomly chosen from T
- **Common GP initialisation: ramped half-and-half, where grow & full method each deliver half of initial population**

Example application: symbolic regression

- **Given some points in R^2, (x_1, y_1), … , (x_n, y_n)**
- **Find function f(x) s.t. $\forall i = 1, \ldots, n : f(x_i) = y_i$**
- **Possible GP solution:**
  - Representation by F = {+, -, /, sin, cos}, T = **R** $\cup$ {x}
  - Fitness is the error $err(f) = \sum_{i=1}^{n} (f(x_i) - y_i)^2$
  - All operators standard
  - pop.size = 1000, ramped half-half initialisation
  - Termination: n "hits" or 50000 fitness evaluations reached (where "hit" is if $| f(x_i) - y_i | < 0.0001$)

4.3 Evolutionary Programming

Typically applied to:

- traditional EP: machine learning tasks by finite state machines
- contemporary EP: (numerical) optimization

Attributed features:

- very open framework: any representation and mutation op's OK
- crossbred with ES (contemporary EP)
- consequently: hard to say what "standard" EP is

Special:

- no recombination!
- self-adaptation of parameters standard (contemporary EP)

Representation

- **For continuous parameter optimization**
- **Consist of two parts:**
  - Object variables: $x_1,\ldots,x_n$
  - Mutation step sizes: $\sigma_1,\ldots,\sigma_n$

Full size: $\langle x_1,\ldots,x_n, \sigma_1,\ldots,\sigma_n \rangle$

Mutation

$$\begin{cases} x_i' = x_i + \sqrt{\sigma_i} \cdot N_i(0,1) \\ \sigma_i' = \sigma_i + \sqrt{\eta \cdot \sigma_i} \cdot N_i(0,1) \end{cases}$$

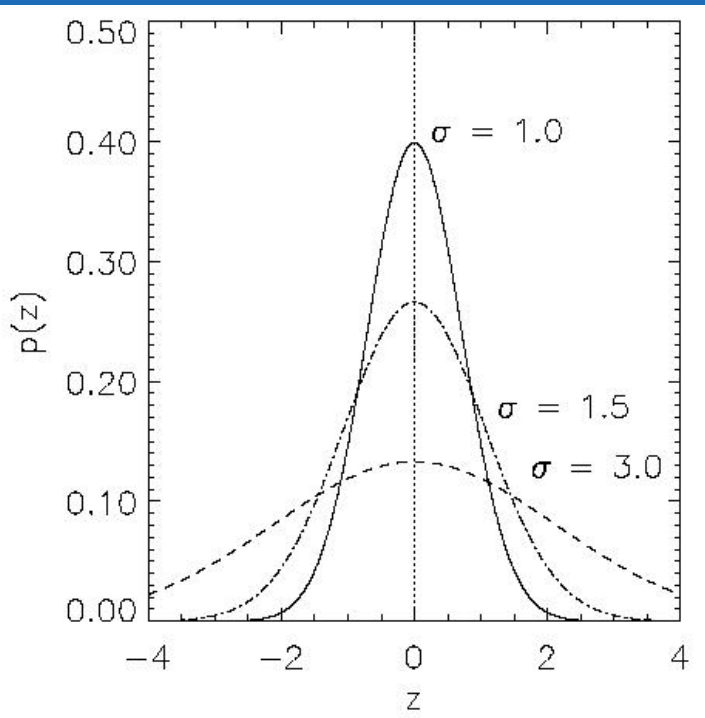


Illustration of normal distribution

- **Other variants proposed & tried:**
  - Lognormal scheme as in ES
  - Using variance instead of standard deviation
  - Mutate σ-last
  - Other distributions, e.g, Cauchy instead of Gaussian

Recombination

- **None**
- **Rationale:**
  - one point in the search space stands for a ***species***, not for an individual and there can be no crossover between species
- **Pragmatic approach seems to prevail today**

Selection

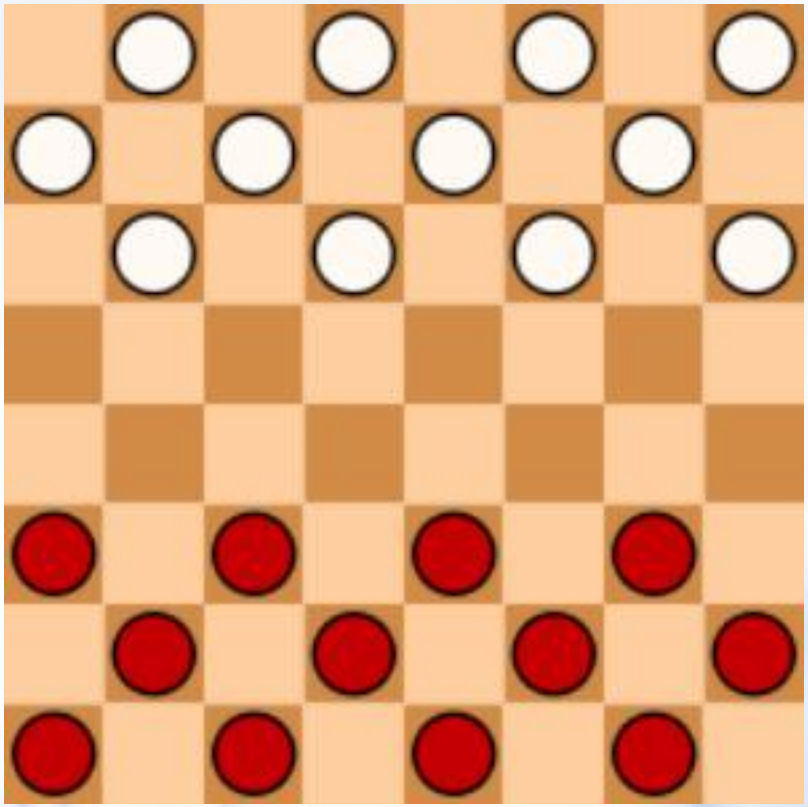
Parent selection

- Each individual creates one child by mutation, thus Deterministic and Not biased by fitness

Survivor selection

- Pairwise competitions in round-robin format:
  - Each solution x from P(t) ∪ P'(t) is evaluated against q (typically q=10) other randomly chosen solutions
  - For each comparison, a "win" is assigned if x is better than its opponent
  - The μ solutions with the greatest number of wins are retained to be parents of the next generation
  - P(t): μ parents, P'(t): μ offspring

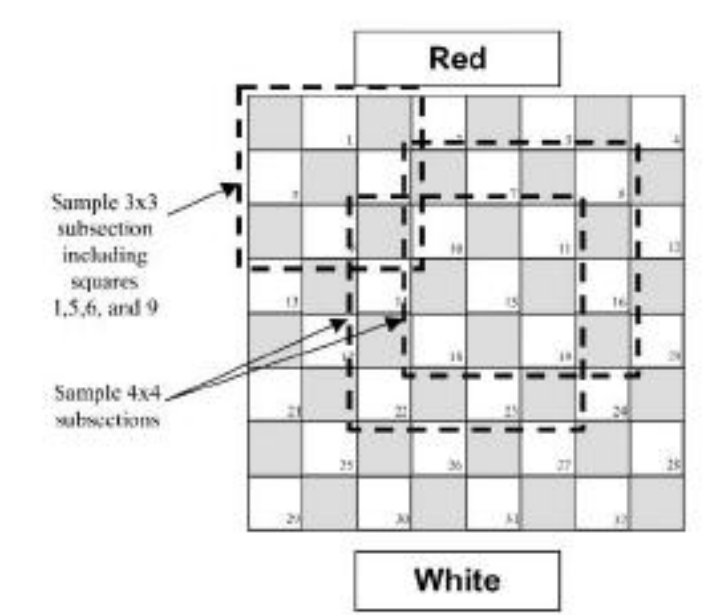
Example application: evolving checkers players [Fogel and Chellapilla, 2002]

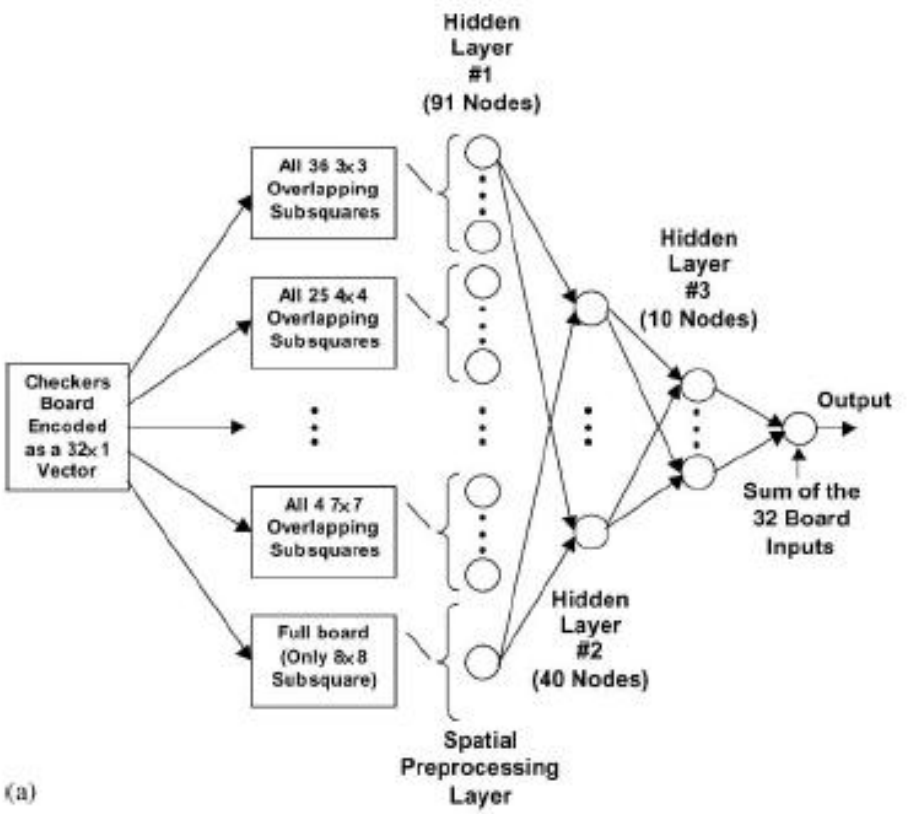


Fogel and Chellapilla, Neurocomputing, 2002, 42: 69-86

Example application: evolving checkers players [Fogel and Chellapilla, 2002]

- **Neural nets** for evaluating future values of moves are evolved
- NNs have fixed structure with **5046 weights**





Example application: evolving checkers players [Fogel and Chellapilla, 2002]

- **Neural Nets are evolved + one value for "kings"**
- **Representation:**
  - vector of 5046 real numbers for object variables (weights)
  - vector of 5046 real numbers for σ's
- **Mutation:**
  - Gaussian, lognormal scheme with σ-first
  - Plus special mechanism for the kings' weight
- **Population size: 15**
- **Tournament size q = 5**

$$\sigma_i'(j) = \sigma_i(j)\exp\left(\tau N_j(0,1)\right),\ j = 1,\cdots,5046$$

$$w_i'(j) = w_i(j) + \sigma_i'(j)N_j(0,1),\ j = 1,\cdots,5046$$

$$K_i' = K_i + \delta$$

Example application: evolving checkers players [Fogel and Chellapilla, 2002]

- Programs (with NN inside) play against other programs, no human trainer or hard-wired intelligence
- After 840 generation (6 months!) best strategy was tested against humans via Internet
- Program earned "expert class" ranking outperforming 99.61% of all rated players.

4.4 Evolutionary Strategies

- **Typically applied to:**
  - numerical optimization
- **Attributed features:**
  - fast
  - good optimizer for real-valued optimization
  - relatively much theory
- **Special:**
  - self-adaptation of (mutation) parameters standard

Representation

- **Consist of three parts:**
  - Object variables: $x_1,\ldots,x_n$
  - Strategy parameters:
    - Mutation step sizes: $\sigma_1,\ldots,\sigma_{n_\sigma}$
    - Rotation angles: $\alpha_1,\ldots,\alpha_{n_\alpha}$
- **Full size:** $\langle x_1,\ldots,x_n,\sigma_1,\ldots,\sigma_n,\alpha_1,\ldots,\alpha_k \rangle$
  - where k = n(n-1)/2 (no. of *i*, *j* pairs)

Mutation

- **Main mechanism:**
  - changing value by adding random noise drawn from normal distribution
- **Self-adaptation:**
  - Mutation step size and rotation angles is coevolving with the solution

$$\begin{cases} \sigma_i' = \sigma_i \cdot \exp\left(\tau' \cdot N(0,1) + \tau \cdot N_i(0,1)\right) \\ \alpha_{ij}' = \alpha_{ij} + \beta \cdot N_j(0,1) \\ x_i' = x_i + z_i \end{cases}$$

Recombination

	Two fixed parents	Two parents selected for each i
z_i is x_i or y_i chosen randomly	Local Discrete	Global Discrete
$z_i = (x_i + y_i)/2$	Local Intermediary	Global Intermediary

- Parent: $x=\{x_1,\cdots, x_n\}$; $y=\{y_1,\cdots, y_n\}$
- Child: $z=\{z_1,\cdots, z_n\}$

Selection

- **Parents are selected**
  - by uniform random distribution whenever an operator needs one/some
- **Survivor Selection:**
  - Applied after creating λ children from the μ parents by mutation and recombination
  - Deterministically chops off the "bad stuff"
    - The set of children only: (μ,λ)-selection
    - The set of parents and children: $(\mu+\lambda)$-selection

Example：Evolution of a Two-Phase Jet Nozzle [Hans-Paul Schwefel]



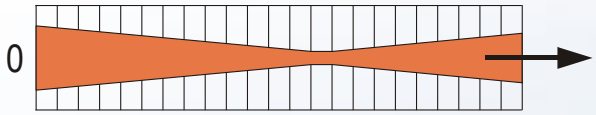
Starting Nozzle (55%)

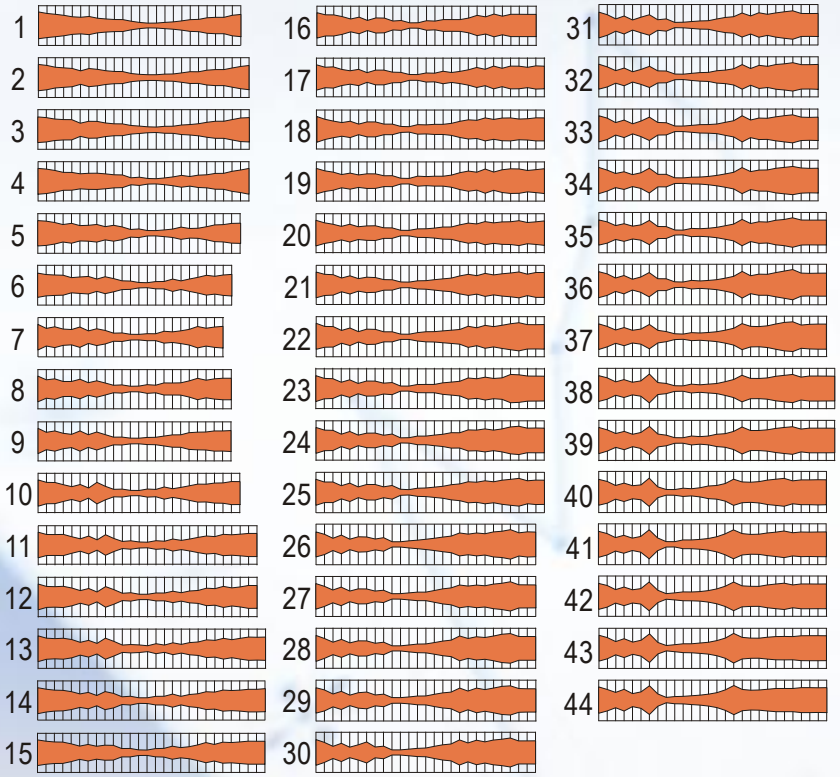


Optimized Nozzle (80%)

- **Goal**
  - obtain maximum thrust
- **Nozzle represented**
  - by a series of conical segments
- **Segments replaced**
  - by rules of 1+1-ES

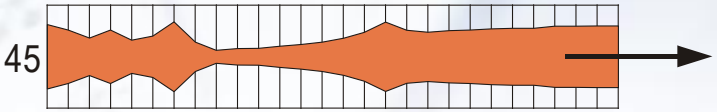
Example: Evolution of a Two-Phase Jet Nozzle [Hans-Paul Schwefel]





The animation of the evolution





BIT.MLMR

End of Lecture 6

Do you have new views about the evolution, genetics and computing?

LIUXIABI@BIT.EDU.CN





